Supplementary Material for: Frontier and Localhost: A Survey of How Production AI Learns Outside the Weights

October 2026

This document accompanies the main article. S1 is the released score table; S2–S5 carry the verification material behind the survey (the data appendix, the mechanism stack, the contrast set and matrix membership, and the walkthrough of the convergent research families); S6 audits every absolute claim of the first-pass draft against the full corpus and holds the rubric-reward comparison that Section IV of the main text summarizes; S7 is the extended related work, organized by the article's four design commitments. Section numbers in the form "Section N" refer to the main article.

Key to the labels used throughout. The substrate codes S1 to S6 name the six scaffold substrates of the main article (S1 instructions, S2 skills, S3 memory, S4 tools, S5 orchestration, S6 governance) and are distinct from the section labels S1 to S7 of this document; INT marks a row scored across several substrates. C1 to C4 are the four feedback-signal classes of Section IV: exact, observable outcome, judged, and delayed and sparse. A system's two gates are written [Loop 1 × Loop 2] with the values Auto, Human, or None; Auto+Human marks a two-stage local gate, an algorithmic check followed by a reviewer. Stages 1 to 3 are the three stages of scaffold maintenance of Table II-1: patchwork, random artifact search, artifact-layer descent. Scopes nest USER within PROJECT within STACK within TENANT within CORE, where a tenant is one organization or customer; promotion that reaches CORE crosses tenants. Properties (a) to (d) are the four properties of the missing architecture in Section IX, and the five conditions are those of Section III. A system named without a citation is a row of the score table in Section S1, which gives its source.

S1. Full score table

Every scored (substrate, system) row of the corpus: 130 systems scored by May 2026 and 126 added by the October 2026 sweep. Scores follow the 0–5 rubric of the main text (Table V-1); Loop 1 / Loop 2 gate types and signal classes are recorded for the October rows (and for May rows re-audited in Section VII). Substrate codes: INT integrated, S1 instructions, S2 skills, S3 memory, S4 tools, S5 orchestration, S6 governance. A year of "—" marks a classical system that predates the survey window. The CSV files with per-row justifications accompany this document.

Sub.	System	Score	Year	Source	Loop 1	Loop 2	Class	Added
INT	ADAS / Meta Agent Search	4	2024	arXiv:2408.08435				May 26
INT	Agent S / S2	4.5	2024	arXiv:2410.08164				May 26
INT	Agent Workflow Memory (AWM)	3.5	2024	arXiv:2409.07429				May 26
INT	AlphaEvolve	4	2025	arXiv:2506.13131				May 26
INT	AutoAgent	5	2026	arXiv:2603.09716				May 26
INT	AutoManual	3.5	2024	NeurIPS 2024				May 26
INT	AutoSaddler	4	2026	arXiv:2608.23041	Auto	None	C2	Oct 26
INT	CaptainAgent	2.5	2024	arXiv:2405.19425				May 26
INT	CASCADE	4.5	2025	arXiv:2512.23880				May 26
INT	ChatDev	3	2023	arXiv:2307.07924				May 26
INT	Co-Harness	4	2026	arXiv:2607.22688	Auto	None	C1,C2	Oct 26
INT	CoEvoSkills	3.5	2026	arXiv:2604.01687				May 26
INT	Cognition Devin	3	2024	cognition.ai				May 26
INT	Continual Harness	4	2026	arXiv:2605.09998	None	None	C4	Oct 26
INT	Cradle	4	2024	arXiv:2403.03186				May 26

Sub.	System	Score	Year	Source	Loop 1	Loop 2	Class	Added
INT	Darwin Gödel Machine	4	2025	arXiv:2505.22954				May 26
INT	Double Ratchet	4	2026	arXiv:2607.12790	Auto	None	C3 anchored by C1	Oct 26
INT	EvolveNet	4	2026	arXiv:2608.04968	Auto	Auto	C1,C2	Oct 26
INT	EvolveR	4	2025	arXiv:2510.16079				May 26
INT	ExpeL	3.5	2023	arXiv:2308.10144				May 26
INT	Generative Agents	4	2023	arXiv:2304.03442				May 26
INT	GenericAgent	4	2026	arXiv:2604.17091	Auto	None	C2	Oct 26
INT	GITM	3	2023	arXiv:2305.17144				May 26
INT	HarnessBank	4	2026	arXiv:2607.13683	Auto	None	C2	Oct 26
INT	HarnessX	4	2026	arXiv:2606.14249	Auto	None	C2	Oct 26
INT	Hippocratic AI Polaris	3.5	2024	hippocraticai.com				May 26
INT	JARVIS-1	3.5	2023	arXiv:2311.05997				May 26
INT	JIT-Agent	4	2026	arXiv:2608.25593	Auto	None	C2	Oct 26
INT	Life-Harness	4	2026	arXiv:2605.22166	Auto	None	C2	Oct 26
INT	Mendel Gödel Machine (MGM)	4	2026	arXiv:2608.07645	Auto	None	C1	Oct 26
INT	MetaGPT	3.5	2023	arXiv:2308.00352				May 26
INT	Mobile-Agent-E	4.5	2025	arXiv:2501.11733				May 26
INT	ModularRSI	4	2026	arXiv:2609.14857	Auto	None	C2	Oct 26
INT	NanoResearch	5	2026	arXiv:2605.10813				May 26
INT	OpenCore	4	2025	github.com/sibmike/opencore				May 26
INT	OpenHands / CodeAct	3	2024	arXiv:2407.16741				May 26
INT	OS-Copilot / FRIDAY	4	2024	arXiv:2402.07456				May 26
INT	Plan4MC	2.5	2023	GitHub/arXiv				May 26
INT	SAGE	4	2025	arXiv:2512.17102				May 26
INT	Sakana AI Scientist	3	2024	arXiv:2408.06292				May 26
INT	ScholarEvolve	4	2026	arXiv:2609.40169	Auto	None	C2	Oct 26
INT	SIA	4	2026	arXiv:2605.27276	Auto	None	C2	Oct 26
INT	Sierra Agent OS	4	2024	sierra.ai				May 26
INT	Sierra Agent OS 2.0	3	2025	sierra.ai/blog/agent-os-2-0				May 26
INT	SkillForge	4	2026	arXiv:2604.08618				May 26
INT	SkillMAS	4	2026	arXiv:2605.09341	Auto	None	C2	Oct 26
INT	SkillOS	3	2026	SemanticScholar: 2653e4c7				May 26
INT	SkillRL	5	2026	arXiv:2602.08234				May 26
INT	TTHE	4	2026	arXiv:2607.08124	Auto	None	C3	Oct 26
INT	VACE	4	2026	arXiv:2609.37105	Auto	None	C2	Oct 26
INT	Voyager	4	2023	arXiv:2305.16291				May 26
INT	Workspace Optimization (DreamTeam)	3	2026	arXiv:2605.09650	Auto	None	C1	Oct 26
S1	AGENTS.md	2	2026	arXiv:2601.20404				May 26
S1	Aider Conventions	2	2024	aider.chat				May 26
S1	Claude Code CLAUDE.md	4	2025	docs.anthropic.com				May 26
S1	Cody / Sourcegraph	2	2024	sourcegraph.com				May 26
S1	Continue.dev Rules	3	2024	docs.continue.dev				May 26
S1	Cursor Rules	3	2024	docs.cursor.com/rules				May 26
S1	GitHub Copilot Custom Instructions	3	2024	docs.github.com				May 26
S1	GRACE	4	2026	arXiv:2607.09175	Auto	None	C2	Oct 26
S1	Lovable Knowledge	2	2024	docs.lovable.dev				May 26
S1	Meta Context Engineering (MCE)	4	2026	arXiv:2601.21557	Auto	None	C2	Oct 26
S1	Replit replit.md	4	2025	docs.replit.com				May 26
S1	Spotify conversational-recommendation loop	4	2026	arXiv:2609.30297	Auto	Human	C3	Oct 26
S1	VS Code Copilot Instructions	3	2024	code.visualstudio.com				May 26
S1	Windsurf Cascade Rules	3	2024	docs.windsurf.com				May 26
S1	Zed AI Rules	3	2024	zed.dev/docs				May 26
S2	AgentVerse	1	2023	arXiv:2308.10848				May 26
S2	Anthropic Claude Agent Skills	3	2025	anthropic.com/engineering				May 26
S2	Anthropic/skills GitHub repo	2	2025	github.com/anthropics/skills				May 26
S2	ASSAY	4	2026	arXiv:2606.15390	Auto	None	C2	Oct 26
S2	AutoSkill	3	2026	arXiv:2603.01145	Auto	None	C3	Oct 26
S2	Buildrix	3	2026	arXiv:2606.25139	None	Human	C3	Oct 26
S2	ChatGPT Plugins (legacy)	1	2023	openai.com				May 26
S2	Claude Code Skills	3	2025	docs.anthropic.com				May 26
S2	COSPLAY	5	2026	arXiv:2604.20987				May 26
S2	CrewAI Skills	2	2025	docs.crewai.com				May 26
S2	HiSME	4	2026	arXiv:2605.28390	Auto	None	C1,C3	Oct 26

Sub.	System	Score	Year	Source	Loop 1	Loop 2	Class	Added
S2	HuggingFace Agent Skills	2	2025	huggingface.co				May 26
S2	LangChain Skills	2	2025	langchain.com				May 26
S2	Library Drift curator	4	2026	arXiv:2605.19576	Auto	None	C1	Oct 26
S2	Meta Context Engineering (MCE)	4	2026	arXiv:2601.21557	Auto	None	C2	Oct 26
S2	MetaGPT (SOP-as-skill)	3	2023	arXiv:2308.00352				May 26
S2	MetaSkill-Evolve	4	2026	arXiv:2607.05297	Auto	None	C2	Oct 26
S2	Microsoft Semantic Kernel Plugins	2	2024	learn.microsoft.com				May 26
S2	MUSE-Autoskill	4	2026	arXiv:2605.27366	Auto	None	C1,C2	Oct 26
S2	NanoResearch Skill Bank	4	2026	arXiv:2605.10813				May 26
S2	OpenAI Custom GPTs / Actions	2	2023	platform.openai.com				May 26
S2	Personalized developer skills (Huang-Du-Lan)	3	2026	arXiv:2608.10319	Auto	None	C3	Oct 26
S2	RASO	4	2026	arXiv:2609.38024	Auto	None	C2	Oct 26
S2	SAGE	5	2025	arXiv:2512.17102				May 26
S2	SCAFFOLD	4	2026	arXiv:2609.05511	Auto	None	C2	Oct 26
S2	SE-GoS	4	2026	arXiv:2609.08228	Auto	None	C2	Oct 26
S2	SkeMex	4	2026	arXiv:2606.09365	Auto	None	C2	Oct 26
S2	Skill Issues (Bauplan)	4	2026	arXiv:2606.01185	Auto	None	C1,C3	Oct 26
S2	SKILL.md smells (Hong-Imani-Ahmed)	1	2026	arXiv:2607.01456	None	None	-	Oct 26
S2	SkillAudit	4	2026	arXiv:2606.14239	Auto	None	C3,C1-partial	Oct 26
S2	SkillBoost	4	2026	arXiv:2607.26643	Auto	None	C1	Oct 26
S2	SkillChain	4	2026	arXiv:2606.12984	Auto->Huma	None	C3,C2	Oct 26
S2	SkillComposer	4	2026	arXiv:2606.06079	Auto	None	C1,C2	Oct 26
S2	SkillEvoReg	4	2026	arXiv:2609.30861	Auto	None	C1,C2	Oct 26
S2	SkillFoundry	3	2026	arXiv:2604.03964	Auto	None	C1	Oct 26
S2	SkillGen	4	2026	arXiv:2605.10999	Auto	None	C2	Oct 26
S2	SkillGLoW	4	2026	arXiv:2609.02217	Auto	None	C2	Oct 26
S2	SkillLift	4	2026	arXiv:2609.15396	Auto	None	C3 inner, C2 outer	Oct 26
S2	SkillOps	4	2026	arXiv:2605.13716	Auto	None	C2	Oct 26
S2	SkillSmith	4	2026	arXiv:2606.01314	Auto	None	C1,C2	Oct 26
S2	SkillSpec (consensus-gated)	4	2026	arXiv:2610.00704	Auto	None	C1,C2	Oct 26
S2	SuperAGI Toolkits	1	2024	github.com/TransformerOptimus				May 26
S2	TRUSS	3	2026	arXiv:2608.17588	Auto	None	C1,C2	Oct 26
S2	VALVE	4	2026	arXiv:2609.32990	Auto	None	C1	Oct 26
S2	VCE-Skill	4	2026	arXiv:2608.16544	Auto	None	C1,C2	Oct 26
S2	Voyager	4	2023	arXiv:2305.16291				May 26
S2	Web2BigTable	3	2026	arXiv:2604.27221	Auto	None	C1,C2	Oct 26
S2	Who Maintains Agent Skills? (empirical)	2	2026	arXiv:2609.05677	None	Human	C3	Oct 26
S2	WikiSkill	4	2026	arXiv:2608.27454	Auto	None	C2	Oct 26
S2	XRepoSkill	4	2026	arXiv:2609.36807	Auto	Auto-parti	C1	Oct 26
S3	A-MEM	3	2025	NeurIPS 2025				May 26
S3	Agent-native memory system evaluation (Zhou et a	2	2026	arXiv:2606.24775	n/a	n/a	n/a	Oct 26
S3	AIM privacy-aware interoperable memory (Johnson	3	2026	arXiv:2609.12320	Auto	None	C3	Oct 26
S3	Beyond the Harness (enterprise Text-to-SQL cards	3	2026	arXiv:2608.22830	Auto	None	C2	Oct 26
S3	ChatGPT Memory	3	2024	openai.com				May 26
S3	Claude Memory	3	2025	anthropic.com				May 26
S3	Cognee	3	2024	github.com/topoteretes/cognee				May 26
S3	Compiled Memory (Rhodes & Kang)	4	2026	arXiv:2603.15666	Auto-gated	None	C1	Oct 26
S3	Control-plane placement / ForgetEval (Yang)	2	2026	arXiv:2606.15903	n/a	n/a	n/a	Oct 26
S3	FRESH failure-aware heterogeneous graph memory (	4	2026	arXiv:2609.28003	Auto	None	C2	Oct 26
S3	GateMem multi-principal memory-governance benchm	2	2026	arXiv:2606.18829	n/a	n/a	n/a	Oct 26
S3	Gemini Code Assist Memory	4	2025	cloud.google.com				May 26

Sub.	System	Score	Year	Source	Loop 1	Loop 2	Class	Added
S3	Generative Agents	3	2023	arXiv:2304.03442				May 26
S3	Governed Collaborative Memory (Cuadros et al.)	5	2026	arXiv:2605.04264				May 26
S3	HippoRAG	2	2024	arXiv				May 26
S3	Honest Lying mitigation / programmatic failure s	4	2026	arXiv:2605.29463	Auto	None	C1/C2	Oct 26
S3	KV-cache rollback consistency (Zhang & Yang)	2	2026	arXiv:2608.15939	n/a	n/a	n/a	Oct 26
S3	LangChain Primitives (legacy)	1	2023	langchain.com				May 26
S3	LangGraph Long-Term Memory	3	2024	langchain.com				May 26
S3	LangGraph Store / LangMem (2026 refresh)	3	2026	docs.langchain.com/oss/python	None	None	n/a	Oct 26
S3	Learning on the Job	4	2026	arXiv:2607.22157	Auto+ Human	None	C2,C3	Oct 26
S3	Letta MemFS + Context Repositories + Dreaming (2	4	2026	docs.letta.com/letta-code	Auto	None	C3	Oct 26
S3	Mem0	3	2025	arXiv				May 26
S3	Mem0 Platform (2026 refresh: history, feedback,	3	2026	docs.mem0.ai/api-reference	Auto	None	C3	Oct 26
S3	MemAudit post-hoc poisoned-memory audit (Tan et	3	2026	arXiv:2605.23723	Auto	None	C2	Oct 26
S3	MemClaw / Governed Shared Memory (Margalit et al	3	2026	arXiv:2606.24535	Auto	None	C3	Oct 26
S3	MemGPT / Letta	3	2023	arXiv:2310.08560				May 26
S3	MemGuard persisted verifier signals (Wang et al.	4	2026	arXiv:2608.21867	Auto-gated	None	C3	Oct 26
S3	MemoryBank	3	2023	Zhong et al.				May 26
S3	Mi-Memory lifecycle framework (Liu et al.; Xiaom	3	2026	arXiv:2607.18975	Auto	None	C3	Oct 26
S3	MIRIX	3	2025	arXiv				May 26
S3	MPBench memory-poisoning study (Dash et al.)	2	2026	arXiv:2606.04329	n/a	n/a	n/a	Oct 26
S3	NanoResearch Memory Module	3	2026	arXiv:2605.10813				May 26
S3	PPMF provenance-preserving memory firewall (Xu e	3	2026	arXiv:2607.29167	Auto	None	C1	Oct 26
S3	REALM retrieval-driven memory reconsolidation (S	4	2026	arXiv:2609.16053	Auto	None	C2/C3	Oct 26
S3	Recuris recursive experiential-working memory (Y	4	2026	arXiv:2608.24876	Auto-gated	None	C2	Oct 26
S3	Reflexion	4	2023	arXiv:2303.11366				May 26
S3	Revocation guard / Revoked but Still Authoritati	3	2026	arXiv:2609.08258	None	None	C1	Oct 26
S3	Safe to Resume? checkpoint-rollback continuity (	2	2026	arXiv:2608.29381	n/a	n/a	n/a	Oct 26
S3	ShareMem	4	2026	arXiv:2609.32511	Auto	Auto-parti	C3	Oct 26
S3	SIx Harness / Memory as Infrastructure (Helwig)	3	2026	arXiv:2609.05510	Auto	None	C2	Oct 26
S3	SOAR Cognitive Architecture	4	—	Newell/Laird				May 26
S3	SSGM stability-and-safety governed memory (Lam e	2	2026	arXiv:2603.11768	None	None	n/a	Oct 26
S3	Stale constraints in inherited memory (Nakayashi	2	2026	arXiv:2608.25553	n/a	n/a	n/a	Oct 26
S3	TMA-NM origin-bound memory authority (Louck)	3	2026	arXiv:2606.24322	Auto	None	C1	Oct 26
S3	TOKI bitemporal operator algebra (Wang)	3	2026	arXiv:2606.06240	Auto or Hu	None	C3	Oct 26

Sub.	System	Score	Year	Source	Loop 1	Loop 2	Class	Added
S3	Trust-scored memory sanitization (Devarangadi Su	2	2026	arXiv:2601.05504	Auto gate	None	n/a	Oct 26
S3	Typed provenance + assertion guardrails / Stored	3	2026	arXiv:2609.02127	Auto	None	C1	Oct 26
S3	Utility Under Attack (Karunanidhi)	2	2026	arXiv:2608.21230	n/a	n/a	n/a	Oct 26
S3	VerificAgent human-verified frozen memory (Nguye	3	2025	arXiv:2506.02539	Human	None	C3	Oct 26
S3	VerMem verifiable memory management (Sun et al.)	4	2026	arXiv:2608.03137	Auto	None	C2/C3	Oct 26
S3	Voyager Skill Library	4	2023	arXiv:2305.16291				May 26
S3	Zep / Graphiti	3	2025	arXiv				May 26
S3	Zep / Graphiti bitemporal facts (2026 refresh)	3	2026	help.getzep.com	Auto	None	C3	Oct 26
S4	Anthropic Tool Use API	4	2024	docs.anthropic.com				May 26
S4	Browser Tools (Playwright/Computer Use)	3	2024	anthropic.com				May 26
S4	Claude Code Tool API	4	2024	docs.anthropic.com				May 26
S4	Code Interpreter	3	2023	openai.com				May 26
S4	Cursor MCP Integration	4	2024	docs.cursor.com				May 26
S4	Devin / SWE-agent Tool Harness	4	2024	cognition.ai				May 26
S4	Gemini Function Calling	3	2024	ai.google.dev				May 26
S4	GPT Actions	3	2023	platform.openai.com				May 26
S4	Harvey (Legal Vertical)	5	2024	harvey.ai				May 26
S4	Hippocratic AI	5	2024	hippocraticai.com				May 26
S4	LangChain Tools Abstraction	3	2023	langchain.com				May 26
S4	LangGraph Tool Nodes	4	2024	langchain.com				May 26
S4	LlamaIndex Tool Ecosystem	4	2023	llamaindex.ai				May 26
S4	Model Context Protocol (MCP)	5	2024	modelcontextprotocol.io				May 26
S4	OpenAI Assistants API	3	2023	platform.openai.com				May 26
S4	OpenAI Function Calling	3	2023	platform.openai.com				May 26
S4	Sierra	4	2024	sierra.ai				May 26
S4	SkillSmith	4	2026	arXiv:2606.01314	Auto	None	C1	Oct 26
S4	Toolformer	2	2023	arXiv:2302.04761				May 26
S4	ToolLLM / ToolBench	3	2023	arXiv:2307.16789				May 26
S5	AgentConductor	0	2026	arXiv:2602.17100	None	None	C2	Oct 26
S5	AgentGrad	4	2026	arXiv:2609.08572	Auto	None	C2	Oct 26
S5	AgentVerse	2	2023	arXiv:2308.10848				May 26
S5	AutoGen / MagenticOne	3	2024	arXiv:2308.08155				May 26
S5	AWS Bedrock Multi-Agent	2	2024	aws.amazon.com				May 26
S5	Bilevel Autoresearch	4	2026	arXiv:2603.23420	Auto	None	C1	Oct 26
S5	CAMEL	1	2023	arXiv				May 26
S5	ChatDev	2	2023	arXiv:2307.07924				May 26
S5	ContextEvo	4	2026	arXiv:2609.34649	Auto	None	C2	Oct 26
S5	Control System for Frozen Agents (Paul)	4	2026	arXiv:2607.25415	Auto	None	C2	Oct 26
S5	CrewAI	3	2025	docs.crewai.com				May 26
S5	Evo2Team	4	2026	arXiv:2609.34135	Auto	Auto	C2	Oct 26
S5	Growing Harness	4	2026	arXiv:2609.26760	Auto	None	C2	Oct 26
S5	HARBOR	4	2026	arXiv:2604.20938	Auto	None	C2	Oct 26
S5	HarnessForge	4	2026	arXiv:2606.01779	Auto	None	C2	Oct 26
S5	Inspect AI	2	2024	ukgovernmentbeis. github.io				May 26
S5	Institutional MAS architectures (Fei et al.)	1	2026	arXiv:2604.27691	None	None	C2	Oct 26
S5	LangGraph	3	2024	langchain.com				May 26
S5	Letta	3	2025	letta.com				May 26
S5	LlamaIndex Workflows	2	2024	llamaindex.ai				May 26
S5	Memento-Skills	4	2026	arXiv				May 26
S5	MetaGPT	2	2023	arXiv:2308.00352				May 26
S5	Microsoft Semantic Kernel	2	2024	learn.microsoft.com				May 26
S5	Multi-Agent Evolve (MAE)	4	2025	arXiv				May 26
S5	OpenAI AgentKit	4	2025	openai.com				May 26
S5	OpenAI Swarm	1	2024	openai.com				May 26
S5	OpenHands / CodeAct	3	2024	arXiv:2407.16741				May 26
S5	OptiMAS	4	2026	arXiv:2608.21918	Auto	None	C2	Oct 26

Sub.	System	Score	Year	Source	Loop 1	Loop 2	Class	Added
S5	PluginRSI	4	2026	arXiv:2609.32423	Auto	None	C2	Oct 26
S5	Pydantic AI	2	2024	pydantic.dev				May 26
S5	Recursive Harness Self-Improvement (RHI)	4	2026	arXiv:2607.15524	Auto	None	C3	Oct 26
S5	Skill-MAS (meta-skill)	4	2026	arXiv:2606.18837	Auto	None	C2	Oct 26
S5	TACO	4	2026	arXiv:2604.19572	Auto	None	C2	Oct 26
S5	Task-CoEvolve	4	2026	arXiv:2608.20169	Auto	None	C2	Oct 26
S5	TOCOMAS	4	2026	arXiv:2609.37953	Auto	None	C2	Oct 26
S5	Vertex AI Agent Builder	4	2024	cloud.google.com				May 26
S5	Workflow-Designing Agents (WDA)	4	2026	arXiv:2609.32458	Auto	None	C2	Oct 26
S6	Agenta	3	2024	agenta.ai				May 26
S6	Agentic Configuration Management ACM (Quessada-V	3	2026	arXiv:2608.11166	None	Human	n/a	Oct 26
S6	AGENTS.md / AAIF	5	2025	agents.md				May 26
S6	Anthropic skills repo	4	2025	github.com/anthropics/ skills				May 26
S6	Azure API Center MCP registry	3	2026	learn.microsoft.com	None	Human	-	Oct 26
S6	Braintrust	5	2024	braintrust.dev				May 26
S6	Cleric.ai	2	2025	cleric.ai				May 26
S6	Competence Not Accuracy	2	2026	arXiv:2608.18719	None	None	-	Oct 26
S6	Constitutional AI	2	2022	Anthropic				May 26
S6	Cursor Rules	3	2024	cursor.com				May 26
S6	Decagon	2	2026	decagon.ai				May 26
S6	DSPy compile	3	2024	dspy.ai				May 26
S6	EvolveNet	4	2026	arXiv:2608.04968	Auto	Auto	C2	Oct 26
S6	EvoUndo	4	2026	arXiv:2608.28363	Auto	None	C1	Oct 26
S6	From Prompts to Contracts	2	2026	arXiv:2607.08028	None	None	C1	Oct 26
S6	Future AGI prompt lifecycle	4	2026	futureagi.com/blog	Auto	Auto	C3	Oct 26
S6	GitHub Copilot enterprise	4	2026	github.com				May 26
S6	Governance conversion (Davis et al. case study)	3	2026	arXiv:2607.01087	Human	None	C2	Oct 26
S6	Governed Capability Evolution (Qin et al.)	3	2026	arXiv:2604.08059	None	Auto	C2	Oct 26
S6	HackProbe	3	2026	arXiv:2609.04665	Auto	None	C1	Oct 26
S6	Inspect AI	3	2024	ukgovernmentbeis. github.io				May 26
S6	LangFuse	3	2024	langfuse.com				May 26
S6	LangSmith Hub	5	2024	smith.langchain.com				May 26
S6	MCP Gateway & Registry (agentic-community)	3	2026	github.com/ agentic-community	None	Human	C1	Oct 26
S6	NanoResearch	2	2026	arXiv:2605.10813				May 26
S6	Official MCP Registry	2	2026	modelcontextprotocol.io	None	None	-	Oct 26
S6	OpenAI AgentKit Evals	3	2025	openai.com				May 26
S6	OPRO	2	2023	arXiv				May 26
S6	Organizational Control Layer (Shi et al.)	2	2026	arXiv:2606.04306	None	None	n/a	Oct 26
S6	Registry-Governed Agent Lifecycle / EDDOps on AW	3	2026	arXiv:2607.00345	None	Auto+ Human	C3	Oct 26
S6	Reward-Free Pairwise Validator	4	2026	arXiv:2607.14408	Auto	None	C3	Oct 26
S6	Runtime Skill Audit	2	2026	arXiv:2606.11671	None	None	-	Oct 26
S6	Runtime-Independent Persistent Agents (Enoch)	3	2026	arXiv:2609.00546	None	None	C1	Oct 26
S6	SafeEvolve	4	2026	arXiv:2608.12851	Auto	None	C3,C2	Oct 26
S6	SAGE statistical acceptance gate (≠ SAGE 2512.17	4	2026	arXiv:2609.36043	Auto	None	C2	Oct 26
S6	Self-Refine	1	2023	Madaan et al.				May 26
S6	Sierra Ghostwriter	2	2026	sierra.ai				May 26
S6	SkillCoach	3	2026	arXiv:2607.01874	Auto	None	C3,C1	Oct 26
S6	SkillProbe	2	2026	arXiv:2603.21019	None	None	-	Oct 26
S6	SkillSpec (intent-masked correctness)	2	2026	arXiv:2609.06052	None	None	-	Oct 26
S6	TextGrad	2	2024	arXiv				May 26
S6	TrueFoundry MCP Gateway	3	2026	truefoundry.com/docs	None	Human	-	Oct 26
S6	Vellum	5	2024	vellum.ai				May 26
S6	Verifier-as-Gatekeeper (VaG)	4	2026	arXiv:2608.05810	Auto	None	C2	Oct 26

Sub.	System	Score	Year	Source	Loop 1	Loop 2	Class	Added
S6	W&B Weave (Prompts)	3	2024	weave-docs.wandb.ai				May 26

S2. Data appendix

Corpus and window. The corpus holds 256 unique systems from January 2023 to the October 2026 cutoff, released as two files. `p3_master_scores.csv` holds the 130 first-pass systems in 140 rows; `p3_refresh_2026-10.csv` holds the 126 second-pass systems in 133 rows, four of which re-audit a first-pass system and replace its row rather than add a system. The corpus therefore has 140 + 133 - 4 = 269 substrate-level rows. A system may be scored on several substrates, so rows outnumber systems, and Table V-2 counts rows.

Two passes. Every system in either pass is scored against one rubric, from its own method section or documentation. First-pass systems were found in arXiv, vendor documentation, and public repositories through May 2026. Second-pass systems, found from June to the October 2026 cutoff, come from six query families: self-evolving skills and skill lifecycle; harness and context optimization; agent memory governance; prompt-as-code and evaluation-gated promotion; federated or multi-tenant skill sharing; and multi-agent topology evolution. Some were reached instead through the citations of the first-pass systems closest to the missing architecture of Section IX. The second pass was a systematic search of the field, not only a search around the known systems, so the per-substrate statistics pool both passes.

Inclusion and exclusion. A system enters the corpus if it either wraps a foundation model with at least one persistent artifact updated from deployment signal or is described in public as "self-improving," "scaffolded," or "agentic." The contrast set enters through the second criterion and is kept on purpose, so that the discriminator has something to mark against (Self-Refine, ReAct, TextGrad, OPRO, engineer-authored convention files; Section S4.1.1). Two kinds of system are excluded: those whose only update mechanism is weight-level fine-tuning or reinforcement learning, which are model-side and belong to the self-evolution literature this survey positions against, and those with no public evidence of deployment or release, such as white papers without code, demos, or a production claim.

Two filters. The patch-local discriminator admits a system to the core corpus when it scores at least 3 on at least one substrate. Of the 256 systems, 204 pass it, 99 of them first-pass. The other 52, 31 of them first-pass, form the contrast corpus: systems described in public as agentic or self-improving that fail the discriminator. Each second-pass contrast row gives its exclusion reason in the released CSV. The stricter composite two-loop audit then asks whether Loop 1 modifies a persisted artifact from session signal and whether Loop 2 has an explicit cross-context promotion mechanism with versioning or lineage. Representative first-pass members of both groups, contrast systems and core systems that fail the composite audit, are listed in Section S4.1.

Systems outside the released table. Fifteen named systems have no row in the released table. Ten are treated at mechanism level in the main text and this supplement: SkillOpt, GEPA, Meta-Harness, CORAL, PACE, MOSS, RCL, FederatedSkill, SkillClaw, and ChronoMem. Each was classified from its paper after the first pass closed and is placed by that reading, in the matrix or among the research families. The other five are contrast or boundary entries, classified from their papers and marked with a dagger in Section S4.1: ReAct, MetaGen, MetaReflection, CLIN, and ACE. The corpus counts (256, 204, 52) exclude all fifteen. SkillOpt is nonetheless one of the eleven research systems that pass the composite audit, because the audit was applied to it from its paper.

Evidence tiers. Each system carries one or more of five tiers: (T1) peer-reviewed publication; (T2) arXiv preprint; (T3) production claim with metrics; (T4) open-source artifact; (T5) industry blog or interview. T1 and T4 weigh most in cluster analysis. T3 counts where the claim is concrete, naming a benchmark or an integration target; T5 counts only when T1–T4 evidence corroborates it elsewhere. Tier flags reach cluster claims through the per-row evidence column of the released CSV, and T3 production claims are never pooled with T1 or T2 evidence in a cluster-level statistic.

Tier distribution. In the first-pass core corpus, about a quarter of the systems carry T1 evidence as their primary tier, roughly half carry T2, and the rest T3 or T4. Preprint-only systems in either pass carry T2,

recorded per row in the released CSV. Some T2 work is recent enough that its ablations may yet be revised; the cluster-level claims survive dropping any single T2 system.

Scoring discipline. Each system is scored substrate by substrate on the survey's 0–5 rubric and also receives an integrated patch-local-adaptation score in {0, 1, 2, 2.5, 3, 3.5, 4, 4.5, 5}. Half-scores are used sparingly, where one criterion is clearly met and another only partly, and ambiguous cases are rounded down. Every score comes from a single rater. The full score table is therefore published, each row with a one-line justification in the released CSVs, so that any ambiguous classification can be checked independently (Section S1).

Composite two-loop criterion (sensitivity analysis). The strict composite audit passes eleven research systems, all first-pass except SkillOpt, and three industry exemplars. Two natural relaxations change that count.

The governance relaxation counts any of versioning, lineage, or rollback as Loop-2 governance. It re-admits MAE, MetaGen, MetaReflection, and CLIN from the first pass, raising the research count to approximately fifteen, and of the second-pass promoters below it admits EvolveNet alone, whose rollback to the previous shared harness counts.

Six second-pass systems promote across users, clients, teams, or repositories with no versioning or lineage on the promoted artifact: EvolveNet, Evo2Team, XRepoSkill, ShareMem, Spotify's recommendation loop, and Future AGI's prompt lifecycle. They fail the audit on governance alone. RASO, which retrieves skills from a public corpus, is not among them, because inbound retrieval is not a promotion (Section VII). A weaker relaxation that asks only for an explicit promotion mechanism, with no governance property at all, would admit five of them, all landing in [Auto × Auto] except Spotify's loop, which would land in [Auto × Human]; none would open a new cell. The sixth, Future AGI's prompt lifecycle, rests on T5 evidence alone, which the tier rule above does not count without corroboration, so it is not admitted under any rule.

The optimizer-state relaxation admits as Loop 1 a session signal that updates optimizer state without persisting an artifact distinct from that state. It re-admits DSPy and TextGrad from the first pass, raising the research count to approximately thirteen. It admits no second-pass system. The second-pass contrast rows are benchmarks, evaluation and attack studies, registries, audits, and conceptual frameworks, plus one orchestrator whose adaptation lives in the weights, and none of them persists optimizer state as its only artifact.

The qualitative findings hold under every relaxation: [Auto × Auto] is well populated; [Human × Auto] and [Human × Human] are empty; [Human × None] holds only curation workflows and [None × Human] only registries, all of which fail the composite audit; the governance-first cluster sits in [None × Auto]; Letta alone passes the audit with an ungated promotion; and no system adds governed cross-tenant promotion.

S3. The mechanism stack and continual-learning notes

S3.1 The mechanism stack

The update rule of Section III expands into six operations that every patch-local system performs, whether by explicit engineering or by accident.

1. **Residual capture.** Log failures, corrections, rejected outputs, user edits, evaluation regressions, and tool-call failures. This is the signal source of Loop 1; without it, z_t has no provenance.
2. **Mode discovery and credit assignment.** Cluster repeated failures into the deployment's own error modes (the catalogue of *The Architecture of Errors* [1], instantiated locally for one patch) and attribute each mode to the scaffold coordinate responsible for it (the correspondence note of the formal appendix).
3. **Delta synthesis.** Turn a recurring mode into a candidate scaffold edit: an instruction rewrite, a new skill, a memory correction, a tool binding, an orchestration change, or an evaluation case. This produces z_t.

4. **Delta validation.** Estimate whether the edit lowers L_D without collateral damage on adjacent inputs. The gate G_t is Auto when the check is an algorithmic criterion and Human when it is reviewer judgment.
5. **Promotion control.** Decide whether the accepted edit stays in the local fork or climbs the scope hierarchy: PROJECT, STACK, TENANT, CORE. This is Loop 2.
6. **Drift and bloat control.** Expire stale rules, prune conflicting memories, roll back regressed skills, and detect overfitting. Without this operation, accumulated edits produce the scaffold-level analogue of plasticity loss.

Each operation is the locus of a separate engineering discipline, and surveyed systems implement them partially and unevenly. Research full-auto systems excel at operations 3 and 4, governance-first systems at operation 5, and production hybrids at most of operations 1 to 5 but rarely at operation 6. Implementation patterns for each operation lie outside this survey.

S3.2 Loop 1 and Loop 2 as descent steps of different radius

Loop 1 proposes and validates local descent steps on a single L_D. Loop 2 widens the update radius: it selects, merges, and promotes accepted local edits so that they apply to a larger set of patch losses averaged together. Acceptance at USER scope descends on a single L_D. PROJECT promotion descends on a loss averaged over the project, and STACK promotion widens the average further. TENANT promotion descends on a tenant-wide average; CORE promotion, the cross-tenant level, descends on an average over tenants. The two-loop design space is therefore more than a taxonomy of engineering patterns: it records *which loss each surveyed system is descending on*, and which estimator gates the step.

S3.3 Overfitting in scaffold adaptation

In weight training, overfitting is usually a defect. In scaffold adaptation, local overfitting is partly the point. Frontier weights are trained to preserve generality across patches, so they must smooth over local conventions and rare workflow-specific failures. A deployment's patch needs the opposite: selective specialization to its own residuals. The scaffold is therefore a layer meant to overfit, but one whose overfitting is scoped, inspectable, versioned, gated, and reversible. A scaffold edit that improves one repository, hospital workflow, or legal diligence process may degrade performance elsewhere. That is not a contradiction; it is patch adaptation. The mistake is promoting that edit beyond its evidence radius.

In non-stationary patches ($D = D_t$), the same descent mechanism tracks a moving optimum rather than converging once. Recency weighting, replay, pruning, expiry, and rollback are therefore not optional hygiene; they are the scaffold-level analogues of continual-learning machinery.

S4. Contrast set, matrix membership, and closest approaches (May 2026 corpus)

The tables in this section describe the first-pass corpus, which closed in May 2026; the second-pass systems that occupy a cell of the Section VII matrix are named in the cell-by-cell discussion that follows the matrix.

S4.1 Contrast set

Two tables mark the two boundaries of the survey. The first lists representative members of the first-pass contrast corpus: 31 systems that fail the patch-local discriminator, scoring below 3 on every substrate. The second lists core systems, each scoring at least 3 on some substrate, that still fail the stricter composite two-loop criterion, so that the asterisked entries of the matrix in Section S4.2.1 can be checked.

Two marks recur. An asterisk, used in the matrix, marks a system that fails at least one criterion of the composite audit and is included for contrast. A dagger has two uses: in the two contrast tables it marks a system classified from its paper because the score table has no row for it (ReAct, MetaGen, MetaReflection,

CLIN, ACE); in the matrix it marks an industry exemplar evaluated with vendor documentation as the primary evidence (Sierra Agent OS 2.0, Cognition Devin).

S4.1.1 Contrast corpus: fails the patch-local discriminator

System	Year	Discriminator failure	Brief reason
Self-Refine [2]	2023	No persistent artifact	In-context iteration only
ReAct [3]†	2023	No persistent artifact	Thought-act-observe loop is intra-session
TextGrad [4]	2024	No artifact distinct from optimizer state	Textual-gradient optimization; SkillOpt [5] (Section VIII; Section S5) adds the persistent `best_skill.md`, held-out validation gate, rejected-edit memory, and textual learning-rate schedule that TextGrad lacks.
OPRO, Toolformer	2023	No persistent deployment artifact	Optimization or tool use inside a single training or inference run
Aider conventions, Lovable knowledge	2024	No feedback loop	Engineer-authored convention files
AGENTS.md (as a standard)	2025	No feedback loop	Cross-vendor artifact format as a workspace file; adoption measured, adaptation absent (Section VI). The specification's governance under the AAIF is a separate S6 entry, "AGENTS.md / AAIF"
LangChain Skills, HuggingFace Agent Skills, CrewAI Skills	2025–2026	No feedback loop	Curated skill packs without failure-triggered update
Tran & Kiela [6]	2026	Not a system	Single-agent baselines match or beat multi-agent under matched thinking-token budgets; constrains S5 claims (see Section VI)

S4.1.2 Core corpus, fails the composite two-loop criterion

System	Year	Composite-criterion failure	Brief reason
Reflexion [7]	2023	No Loop 2	Per-agent episodic memory without promotion
Generative Agents	2023	No Loop 2	Per-agent reflection + importance memory
ExpeL	2023	Loop boundary blurred	Experience bank built from training tasks
Mem0 (vanilla)	2024	No Loop 2	Per-user memory; no cross-context promotion
Zep	2024	No Loop 2	Per-agent/session knowledge graph
ChatGPT Memory	2024	No Loop 2	Per-user persistence
Claude Memory	2025	No Loop 2	Per-conversation-thread persistence
MemGPT	2023	No Loop 2	Virtual-context management per agent
DSPy	2023	No artifact distinct from optimizer state	Prompt-template optimization; SkillOpt [5] (Section VIII; Section S5) realizes the artifact-bearing form.
MetaGen†	2026	No Loop 2; no cross-session persistence	Role pool dynamic within single inference call
MAE (Multi-Agent Evolve)	2025	No Loop 2	Co-evolution within Proposer/Solver/Judge triplet

System	Year	Composite-criterion failure	Brief reason
MetaReflection†	2024	No Loop 2	Per-agent semantic memory
CLIN†	2023	No cross-instance promotion	Causal abstractions per-agent-per-environment
LangGraph, AutoGen, CrewAI	2023–2024	Framework only	Orchestration frameworks; not deployed systems
W&B Weave, Agenta, Langfuse	2024	No Loop 1	Observability / eval-as-code
Mobile-Agent-E, Cradle, OS-Copilot	2024–2025	No Loop 2	Self-evolving but per-session/per-deployment
Agent S, Agent S2	2024–2025	No Loop 2	Computer-use agentic framework
ACE [8]†	2025	No Loop 2 in surveyed form	Single-context evolving playbook
Cursor Rules (vanilla)	2024	No Loop 2	Per-project artifacts
GitHub Copilot custom instructions, Windsurf, Continue.dev, Zed AI rules	2024	No Loop 1	Engineer-authored at repo base branch
Replit `replit.md`	2025	Loop 2 absent	Self-writing scaffold but per-workspace
SkillForge [9]	2026	No Loop 2	A judge flags failed replies that drive each skill rewrite; the rewrite is committed without an acceptance test, versions stay in the scenario deployment, and engineers review replies, not skill versions
AutoAgent [10]	2026	No Loop 2	Per-agent tool and peer descriptions revised on an uncalibrated consistency judgment; never promoted or versioned
SkillRL [11]	2026	No gate on either loop	A success rate below 0.4 triggers skill proposals; the skill bank takes each one as an unconditional union

The contrast systems fail the discriminator in three main ways: they iterate in context only, they have no feedback loop, or they are not systems at all. Core systems fail the composite criterion mostly for want of a Loop 2.

S4.2 Matrix membership and closest approaches

S4.2.1 Two-loop matrix: cell-by-cell discussion

Both axes record who decides, as Section III defines: a gate is Auto when a program applies a check, Human when a person decides, and None when the step is absent or runs unchecked. An engineer-authored or registrant-supplied artifact with no failure trigger therefore has Loop 1 None in both passes, and who supplies each candidate is listed separately in the occupant table at the end of this section.

The first-pass matrix, with the marks of Section S4.1:

	Loop-2 Auto-gated	**Loop-2 Human-gated**	**Loop-2 None**
Loop-1 Auto-gated	NanoResearch, SkillOpt, AlphaEvolve, ADAS, DGM, Voyager, SAGE, EvolveR, CoEvoSkills, AutoManual	CASCADE, Sierra Agent OS 2.0†, Cognition Devin†	SkillForge*, AutoAgent*, MAE*, MetaGen*, MetaReflection*, CLIN*, DSPy*, TextGrad*
Loop-1 Human-gated	[**empty by engineering logic**]	[**empty in the first pass**]	*(none)*
Loop-1 None	ExpeL* (weak), Braintrust*, LangSmith Hub*, Vellum*	Anthropic skills repository*	Self-Refine*, ReAct*, Mem0*, SkillRL*

The matrix places the first-pass systems audited under the composite criterion, with SkillOpt, and the contrast entries marked with an asterisk. Every matrix entry that passes the audit sits in [Auto × Auto] or [Auto × Human]: the ten research systems of the first cell, with CASCADE, are the eleven research systems of Section V, and Sierra Agent OS 2.0 and Cognition Devin are two of its three industry exemplars. The third exemplar, Letta's Context Repositories, is a second-pass system.

Three placements need a word. DSPy and TextGrad sit in [Auto × None] because their Loop 1 is automated but persists only optimizer state, which only the optimizer-state relaxation of Section S2 would count as a persisted artifact. DGM sits in [Auto × Auto] although its local gate is a validity check rather than a loss estimate: the archive admits every child that compiles and can still edit code, the benchmark score weights parent selection rather than deciding retention, and no person decides anything in the loop, since self-modifications are evaluated without human intervention [12]. SkillRL sits in [None × None] although its loop is failure-driven: a per-category success rate below 0.4 triggers new skill proposals, and the skill bank takes each proposal as an unconditional union, so the step that keeps an edit runs unchecked, and the bank never leaves its run.

COSPLAY and Cuadros et al., which are first-pass rows, and PACE, CORAL, GEPA, Meta-Harness, MOSS, and RCL, which were classified from their papers after the first pass closed, carry no composite audit, so they are neither placed in the matrix nor counted in Section V. Second-pass systems record their gate pairs in the released score table, and the paragraphs below name those that occupy a cell.

[**Auto × Auto**] is the most populated cell: ten systems, each with a machine-evaluated criterion on both loops. Speed on both loops is what its members share; cross-organization governance is what every one of them lacks.

[**Auto × None**] holds the systems whose local loop is gated and whose promotion has no gate, whether because nothing is promoted or because everything is. One audit passer sits here: Letta's Context Repositories push shared memory across agents automatically, with git lineage and no evaluation of any kind, so the composite audit admits them on lineage and the matrix records a promotion that nothing decides (Section VII). Mem0 Platform, TOKI, Learning on the Job, and RASO sit here with no promotion at all; RASO retrieves skills from a read-only public corpus, which is inbound retrieval rather than promotion [13]. Two first-pass systems join them, each starred because it has no Loop 2. SkillForge's judge flags the failed replies that drive each skill rewrite, the rewrite is committed without an acceptance test, and every version stays in the scenario deployment; its support engineers review replies, not skill versions. AutoAgent's analyzer accepts revisions to one agent's tool and peer descriptions on an uncalibrated consistency judgment, and nothing leaves that agent's store. VACE belongs here too although its gate is a held-out check: it compares a candidate harness with the incumbent against the freshly trained checkpoint of the same run, so the edit is made and kept inside one training run [14].

[**Auto × Human**] is the production-viable cell. Loop 1 fires at machine speed under an algorithmic criterion, and Loop 2 waits for a person before an edit propagates; the pairing reconciles the speed of automation with a reviewer's accountability. CASCADE, Sierra Agent OS 2.0, and Cognition Devin occupy it: a

human-and-agent consolidation gate, human-authored expert answers, and a reviewer's merge of a pull request each put a person on promotion.

Six second-pass systems promote across users, clients, teams, or repositories with no versioning or lineage on the promoted artifact, so they sit outside the strict audit: EvolveNet, Evo2Team, XRepoSkill, ShareMem, Spotify's recommendation loop, and Future AGI's prompt lifecycle. Were the audit to ask for a promotion mechanism and no governance property at all, five of them would enter, all landing in [Auto × Auto] except Spotify's loop in [Auto × Human], without opening a new cell; the sixth, Future AGI's prompt lifecycle, rests on T5 evidence alone and is not admitted under any rule (Section S2).

[None × Auto] holds the governance-first prompts-as-code systems: Braintrust, LangSmith Hub, and Vellum. Their Loop 2 is real, since evaluation-gated CI blocks a promotion on regression. They have no Loop 1 in the patch-local sense, because engineers author the candidate artifacts at their desks and session feedback never triggers one, and the asterisks record exactly that: governance without inner-loop adaptation. The cell is informative because it shows what survives when only the outer loop is automated. ExpeL, starred and also marked weak, sits here for a different reason: its experience bank is built from training tasks, which blurs the boundary of its loop, so it fails the audit (Section S4.1.2).

[Human × Auto] is empty in both passes: no surveyed system puts a person on the local gate of a failure-driven loop and then promotes the result by program. A plausible occupant would ship person-approved skills through an automatic held-out evaluation; no published system does.

[Human × Human] is empty too. The registries that a first reading puts here (Azure API Center, the agentic-community MCP Gateway and Registry, TrueFoundry, Buildrix) have no failure-driven loop: a person registers an artifact and a person, or an admission scan, approves its use, which is a promotion gate on a registrant-supplied artifact. They sit in **[None × Human]** with the Anthropic skills repository and Agentic Configuration Management, whose lifecycle states a governance role assigns, and the five public skill repositories of the field study (Section VII) show the same shape in practice. Enoch's checkpoint-and-validate protocol moves one agent's state between hosts and is neither loop. The released table records these re-codings (Section S6.7).

[Human × None] is the curation cell, also filled only from the second pass and single-loop by construction. In VerificAgent an agent drafts, a person fact-checks, and the result is frozen; in SkillChain a judge filters the candidate edits and a person decides, at a reflection gate, which refined skills are deployed; in the Davis et al. case study one engineer both writes and accepts the governance mechanisms that failures call for (Section VII). Curation stays patch-local, rather than collapsing into an engineer editing a configuration file, because the candidate answers a failure and a person decides whether it stays.

The occupants, with the variable the matrix does not show:

Cell	Systems	Candidate source	Loop 1 gate	Promotion mechanism	Loop 2 gate
[Auto × Auto]	NanoResearch, SkillOpt, AlphaEvolve, ADAS, DGM, Voyager, SAGE, EvolveR, CoEvoSkills, AutoManual	agent proposals from trajectories or failures	evaluator, verifier, or threshold	merge into a shared bank or archive	the same evaluator, on the shared bank
[Auto × Human]	CASCADE, Sierra Agent OS 2.0, Cognition Devin	agent proposals	calibrated judge or outcome threshold	release to the team or product	reviewer, domain expert, or merge approval

Cell	Systems	Candidate source	Loop 1 gate	Promotion mechanism	Loop 2 gate
[Auto × None]	Letta Context Repositories; Mem0 Platform, TOKI, Learning on the Job, RASO, VACE; SkillForge, AutoAgent	agent (Letta: sleep-time subagents)	automatic write gate (RASO: validation-set acceptance; VACE: held-out comparison against the incumbent harness; SkillForge: a calibrated judge that flags failed replies; AutoAgent: an uncalibrated consistency judgment)	Letta: unconditional git push to shared memory; the others: none	none
[None × Auto]	Braintrust, LangSmith Hub, Vellum, Governed Capability Evolution; EDDOps, with a sign-off after the threshold	engineers	none (no failure-driven loop)	evaluation-gated release	evaluation threshold
[None × Human]	Azure API Center, MCP Gateway and Registry, TrueFoundry, Buildrix, Anthropic skills repository, Agentic Configuration Management	registrants, contributors, engineers	none	admission to a registry, merge into a shared repository, or a lifecycle state	approval by a person (the MCP Gateway adds an automated admission scan)
[Human × None]	VerificAgent, SkillChain, Davis et al.	agent drafts (VerificAgent; SkillChain after a judge); the engineer (Davis et al.)	a person	none	none

S4.2.2 Closest approaches to governed cross-tenant scaffold optimization

Section IX defines the four properties: (a) an auto-gated Loop 1, (b) cross-organization promotion, (c) versioned lineage with access control, and (d) rollback. The closest approaches split them among themselves:

- **AlphaEvolve** has (a) and a partial (b) confined to one organization; it has no mechanism for promotion across organizations.
- **NanoResearch** has (a) at the research level, and its paper describes no enterprise governance layer.

- **SkillOpt** has (a) fully and a partial (c), through its exported `best_skill.md` artifact and protected slow-update field. Its reported positive transfer across models, harnesses, and benchmarks is an empirical measurement of evidence radius. It implements neither (b) nor explicit rollback (d). The walkthrough in Section S5 maps each of its design decisions onto the loop.
- **CORAL** [15] has a partial (a) through reflection-written shared memory and an automatic Loop 2 through heartbeat-based promotion; it has no (b), no formal access control for (c), and no (d). It is the only system classified before the second pass whose Loop 2 runs asynchronously across multiple agents inside one organization, through shared persistent memory. Evo2Team and ShareMem, second-pass systems, promote across teams and across users without versioning.
- **PACE** [16] has (a) at small-model production timescales and a fast/slow two-timescale pair of loops that consolidates across one deployment's own episodes, not across deployments; it has no (b) and no formal (c) or (d). It is the closest demonstration that the two-loop architecture runs under small-frozen-model constraints.
- **Meta-Harness** [17] has (a) over the harness (substrate S5), with a filesystem of candidates that records credit assignment and serves as the accumulated history H_t of Appendix XII. It has no cross-organization promotion, no access control, and no rollback.
- **SkillForge** has a partial (a), a calibrated judge that selects failures with no acceptance test on the rewrite, and a partial (c) through VFS commits, but no (b): each rewritten skill version stays in the scenario deployment that produced it, and its support engineers review replies, not skill versions; (d) is undocumented in the public report.
- **Sierra Agent OS 2.0** has a partial (a). Its cross-organization promotion is claimed in vendor documentation only and not counted as evidence under the tier rule of Section V, and its GitHub-style Workspaces give some lineage but not access control with rollback in the strict sense.
- **FederatedSkill** [18] has (a) and, with EvolveNet (Table IX-1), the strongest partial (b) among the surveyed systems: heterogeneous clients, each a backbone-plus-harness profile under one federation operator, exchange semantic skill diffs, and a server-side evolution agent applies, adapts, or rejects each patch per client. An auditable decision log, recording source patch, reward, and justification for each modified path, gives a partial (c); there is no access control and no (d).
- **SkillClaw** [19] has a partial (a), through a nightly validation step, and a partial (b) across the users of one shared, synchronized skill repository; it reports no (c) or (d).
- **ChronoMem** [20] has (d) and the versioning half of (c) for the memory substrate alone: whole-memory snapshots, version histories, and natural-language rollback inside Google's Agent Development Kit. It has no Loop 1, no (b), and no access control.
- **Vellum, Braintrust, and LangSmith Hub** have (c) and (d) explicitly but neither (a) nor (b) in the strict sense; they are governance-first by construction.

No surveyed system combines (b), (c), and (d) under any gate type. The cross-organization governance triad is unfilled on its own, whether or not Loop 1 is auto-gated.

S4.2.3 Whether the missing architecture should be built

Whether the missing architecture should be built depends on the feedback-signal class, not on the application domain: safety-critical domains such as medical decision support, legal advice, and financial advice, where a reviewer's gate on promotion is a design feature rather than a defect, are judged or delayed domains with high stakes. Section IX gives the argument class by class.

S5. The convergent research families, system by system

This appendix walks the convergent research families of Section VIII system by system, at the level of mechanism, so that each conclusion of that section can be checked against what the systems do. The walkthroughs were written before the second pass and updated where the audit of Section S6 revised a claim; the members the second pass adds to each family appear in Table VIII-1 and in Section S1.

S5.1 The convergent landscape

SkillOpt [5] is the clearest instance of artifact-layer descent we found, and it is one of at least eighteen independent research threads from 2025–2026 that converge on the same disciplined, gradient-like update for the scaffold layer. They converge from different starting points: GEPA from prompt optimization and Tiwari et al. from continual learning; RCL from cognitive-agent learning; Meta-Harness, AHE, MOSS, CANTANTE, and Ong et al. from harness engineering; MOCHA and Ratchet from skill-deployment constraints; ICT, Combee, CORAL, and PACE from cross-task, multi-agent, and production deployment. Each instantiates a different subset of the five conditions of Section III, and each exposes a different gap.

By substrate (Section V), GEPA and RCL work on the prompt and context layer (S1–S2); the harness family on orchestration (S5), with MOSS reaching into source code (S2+S5); MOCHA and Ratchet on skills under deployment constraints (S2); the transfer family on integrated memory-plus-orchestration stacks (S3+S5); and SkillOpt on an integrated stack led by S2.

No member of any family combines the four properties of Section IX: (a) auto-gated Loop 1, (b) cross-organizational Loop 2, (c) versioned lineage with role-based access control (RBAC), and (d) rollback. The loop of Section III is realized in fragments across the families, and every member leaves the Section IX gap open.

S5.2 SkillOpt: full correspondence

The table maps each component of SkillOpt to its quantity in Section III, with the role that quantity plays and a pointer into the published procedure (Algorithm 1 of the SkillOpt paper).

Section III quantity	SkillOpt instantiation	Section III role explained	Algorithm pointer
θ_s (scaffold)	`best_skill.md` — a 379–1,995-token markdown document	the trainable external state of the frozen target model; persistent and exported as the deployment artifact	s_{best} (Algorithm 1, lines 1, 22, 37)
L_D (patch loss)	benchmark-native hard score / exact-match on a held-out test split	scalar quality signal that the optimizer is implicitly minimizing	$\textsc{Evaluate}(M, h, s, D)$ (lines 2, 17, 37)
Q_t (proposal)	optimizer-model reflection over failure and success minibatches; failures and successes analyzed separately, then merged with priority on failure fixes	proposal distribution conditioned on accumulated rollout evidence and current scaffold	optimizer calls O for failure analysis, success analysis, and merge (lines 9–11)
z_t (edit)	structured add/delete/replace patch on the skill (or rewrite-from-suggestions in rewrite mode)	the candidate update applied via $\oplus$ to the current scaffold	rank/clip to η_t edits, apply to obtain $\tilde{s}$ (lines 12–13)
η_t (textual learning rate)	bounded per-step edit budget; constant / linear / cosine / autonomous schedules; default cosine with floor $\eta_t = 2$	edit-magnitude controller; the step size of Section III in textual form	rank/clip step (line 12); schedule applied per epoch
G_t (gate)	held-out selection-split accept gate, strictly greater than current selection score	the per-edit gate G_t — accepts only when $\widehat{\Delta}_{z_t} L_D$ improves the selection score	comparison at line 20; cache lookup at lines 14–18

Section III quantity	SkillOpt instantiation	Section III role explained	Algorithm pointer
rejected-edit buffer	epoch-local store of failed proposals and the score drop they caused	converts rejected steps into negative feedback for later proposals within the same epoch; zero deployment cost	$\mathcal{B}$ updated at line 26; reset at line 5
slow/meta update	epoch-end longitudinal guidance written to a markup-fenced protected slow-update field, still passed through D_{sel}	second-order consolidation across adjacent epochs; step-level edits cannot overwrite the protected slow-update field	lines 28–31; gate applied to injected guidance
optimizer-side meta skill	m_{meta} — a prompt-side summary of which edits helped, which failed, and which failures persisted	training-time guidance for Q_t; never shipped with the deployed artifact	lines 33–34 (training-side only)

The mapping is one-to-one at the level of Algorithm 1: every Section III quantity has a named, code-level counterpart, and every SkillOpt control surface maps onto a Section III quantity. SkillOpt is therefore the clearest instance we found in which the loop of Section III is *mechanism* rather than *metaphor*: all five conditions are instantiated.

S5.3 What SkillOpt does that prior textual-optimization systems do not

SkillOpt is compared with six baselines under one protocol (same target model, held-out test split, and scorer), and each baseline lacks a specific piece of the loop.

- **TextGrad** [4] backpropagates textual feedback through a compound system's variables but leaves no artifact distinct from optimizer state, which is why Section S4.1 places it outside the core corpus; it has no step bound, held-out gate, or rejected-edit buffer. SkillOpt leads it on direct chat by +5.9 to +34.0 points across benchmarks at GPT-5.5 scale, and at all seven target-model scales.
- **GEPA** [21] evolves prompts by reflection under Pareto-style selection. Its target is a prompt rather than a persistent skill, its selection is not a strictly-greater held-out gate, and nothing bounds an accepted edit, so one edit can rewrite the prompt wholesale. SkillOpt matches or beats it in every reported cell of its main results table [5].
- **Trace2Skill** [22] distils trajectory-local lessons into reusable skills offline. No held-out loop accepts or rejects each candidate, so delta validation (operation 4 of Section S3.1) and the rejected-edit buffer are missing. SkillOpt leads it by +5.0 to +19.0 points at GPT-5.5 direct chat.
- **EvoSkill** [23], the strongest baseline inside the Codex and Claude Code harnesses, evolves a skill folder from failure analysis. Its credit assignment is genuine, but per-step folder edits are unbounded and rejected edits are not kept. SkillOpt leads it by +14.0 inside Codex and +3.2 inside Claude Code on the GPT-5.5 five-benchmark average.
- **A human-written skill** (145–516 tokens per benchmark) persists but never updates: the no-Loop-1 ceiling.
- **A one-shot LLM skill**, generated once by GPT-5.5 from the benchmark description, never updates and proposes without rollout evidence: the untrained-scaffold baseline.

ABSTRAL [24] and EvoTest [25], cited by SkillOpt but not measured, work at the multi-agent-design and test-time-configuration layer and fall outside the comparison.

The live difference across the six is four controls: a persistent edit target distinct from optimizer state (θ_s); a held-out gate on every acceptance (G_t, with estimator $\widehat{\Delta}$ and threshold τ_t); rejected edits kept as negative feedback for Q_t; and a bounded, scheduled edit budget (η_t). SkillOpt is the first system we found that has all four, run as one offline training loop whose deployed artifact, a 379–1,995-token markdown file, needs no optimizer calls at inference.

Its follow-up, SkillOpt-Lite [26], asks which controls are necessary. Recasting skill optimization as zeroth-order search, it keeps three principles: trajectory exploration through the file system, consensus attribute mining (attributes common across heterogeneous rollouts, single-sample quirks discarded), and a held-out gate strictly disjoint from the training failures. It drops minibatch reflection pooling, the rejected-edit buffer, the slow update, and the learning-rate schedule. It still converges faster and scores higher than full SkillOpt (LiveMath +8.8 points at GPT-5.5, +25.4 at GPT-5.4-nano). The gate and a consensus form of credit assignment are load-bearing; negative-feedback retention and the bounded step are stabilizers whose value depends on the benchmark. None of this touches the governance properties of Section IX.

S5.4 GEPA, Tiwari, and the reflective-evolution family

The immediate predecessor to SkillOpt's optimizer framing is **GEPA** [21]. It shows that natural-language reflection over execution trajectories can provide a richer adaptation signal than sparse scalar rewards for prompt evolution in compound AI systems. GEPA samples trajectories containing reasoning steps, tool calls, and tool outputs; reflects on them to diagnose failures and propose prompt updates; and maintains a Pareto frontier of candidate prompts rather than greedily mutating only the current best. Across six tasks, GEPA outperforms GRPO by 6% on average and by up to 20% while using up to 35× fewer rollouts, and also outperforms MIPROv2 in the reported aggregate comparison. In Section III vocabulary, GEPA strongly instantiates the proposal side, Q_t, through trajectory-conditioned reflection, and uses Pareto-based candidate selection as a proposal and pruning mechanism. What it does not instantiate is the full SkillOpt-style artifact lifecycle: a bounded edit budget on a reusable skill document, a strictly-greater held-out gate on each accepted skill update, a rejected-edit buffer used as negative feedback, and an exported deployment artifact with governance hooks.

Tiwari et al. [27] supply the broader framing from the optimization side, arguing that restricting learning to *either* in-context adaptation (prompt optimization) *or* in-weights adaptation (RL or fine-tuning) is unnecessary and costly. In-context adaptation is cheap and rapid but capacity-bounded; in-weights adaptation is expressive but induces catastrophic forgetting and loss of plasticity [28]. The scaffold layer of Section II is the substrate that absorbs what in-context adaptation can carry cheaply, leaving the weights free to generalize. Tiwari et al. reach the two-timescale motivation from optimization and Section II reaches it from production reliability; the two framings are independent and reinforce each other.

S5.5 Reflective Context Learning: a parallel formalization

The most direct independent convergence on the Section III formalization comes from **Reflective Context Learning** [RCL; [29]]. Their opening observation is that "the fundamental problems of learning, including credit assignment, overfitting, forgetting, local optima, and high-variance learning signals, persist whether the learned object lies in parameter space or context space," and that current methods are "fragmented and ad hoc" precisely because they address these challenges one at a time rather than as a unified optimization problem over the context. The parallel to Section III's argument, that scaffold maintenance needs the discipline of weight-space optimization applied to an external artifact layer, is exact and was arrived at independently.

RCL maps onto Section III as follows. The *learned context* is θ_s. The *reflection signal* is a form of credit assignment identifying which context coordinate caused a failure (Section S3.1, operation 2). The *generalization penalty* that RCL introduces to prevent narrow sample-specific rules is structurally the complexity penalty $\Omega(z)$ of the gate condition. Where RCL and Section III differ is in Loop-2 scope: RCL is framed around single-agent cross-task generalization, while Section III's Loop 2 is cross-deployment, cross-tenant, and cross-organizational promotion. The convergence is nonetheless significant: two research threads starting from different motivations (production reliability in Section III, cognitive agent learning in RCL) arrive at the same formal observation that the classical learning problems do not disappear when the learned object moves outside the weights.

S5.6 The harness-optimization cluster

A second family optimizes the *harness*, the code and configuration that wrap a frozen model, rather than skill documents. It names credit assignment as the central unsolved problem of scaffold evolution, and its solutions instantiate two operations of the mechanism stack in Section S3.1: mode discovery with credit assignment (operation 2) and delta validation (operation 4).

Meta-Harness [17] starts from the observation that system performance "depends not only on model weights, but also on their harness", the scaffold-versus-weights decomposition stated for this substrate. Existing text optimizers, it finds, "compress feedback too aggressively" for credit assignment, so it gives an outer-loop proposer filesystem access to the source code, scores, and execution traces of every prior candidate. The edit history becomes the credit-assignment signal. In Section III terms the filesystem is the accumulated history H_t that conditions the proposal Q_t, and the outer loop closes Loop 1 over orchestration (S5) rather than skills (S2).

Agentic Harness Engineering [AHE; [30]] comes at the problem from observability. It names three obstacles: a heterogeneous action space across editable components, voluminous trajectories that bury the actionable signal, and edits whose effect is hard to attribute. They are, in order, coordinate identification (operation 2), residual capture (operation 1), and the credit-assignment half of operation 2. AHE's answer, three observability pillars that give every editable harness component a file-level representation, builds the coordinate decomposition that Section III assumes. Descent needs that instrumentation as much as it needs the loop.

MOSS [31] observes that earlier self-evolving systems "confine evolution to text-mutable artifacts" and leave the harness untouched. Routing, hook ordering, state invariants, and dispatch live in code, so a whole class of structural failure is, in MOSS's phrase, "physically unreachable from the text layer". MOSS therefore extends Loop 1 to rewriting source code, evolving skills (S2) and orchestration (S5) together. The consequence for Section IX: a governed optimizer that covers only text artifacts leaves the code substrate ungoverned.

CANTANTE [32] takes up credit assignment in multi-agent scaffolds, where "scores are available only at the system level, whereas the parameters governing agent behavior are local". It contrasts rollouts of several jointly acting configurations to split a system-level reward into per-agent update signals. That gives the credit-assignment step a concrete mechanism when the scaffold is spread across agents (S5): the attribution, succeeding with non-zero probability per failure, that the correspondence note of the formal appendix requires.

Harness-optimizer evaluation [33] finds that judging scaffold optimizers "solely by observing target agents' performance gains" misses intermediate erroneous edits that hinder performance, and that it is "unclear whether harness optimization is driven by optimizers' informed update actions or simply trial-and-error". That is the line between Stage 2 and Stage 3 of Section III, measured. An optimizer that cannot tell informed updates from lucky ones operates in Stage 2, whatever its benchmark gains.

HARNESSEVO [34] turns the diagnosis into a measurement. It splits the harness into four separately evolvable slots (role, task strategy, tool and format rules, reflection and control) and attributes value by leave-one-in and leave-one-out evolution. On ALFWorld with a 7B target model nearly all of the gain sits in the reflection and control slot (leave-one-in +0.119; the other slots individually null). Spread evenly, a fixed 64-rollout budget leaves every slot below the optimizer's effective search floor, so each freezes at its seed; concentrated on the high-value slot, it recovers the gain (0.642 to 0.761). On WebShop the effect is absent. Credit assignment over coordinates has to precede budget allocation, and which coordinate carries the value depends on the task.

S5.7 Multi-objective and lifecycle constraints

Descent on a single loss misses two kinds of constraint: the platform limits a deployed skill must satisfy, and the hygiene of the artifact pool from which the gate draws its signal.

MOCHA [35] observes that agent skills are "multi-field artifacts subject to hard platform constraints": descriptions are truncated for routing, instruction bodies are compacted by progressive disclosure, and co-resident skills compete for the context window. A skill must maximize task performance and satisfy

those limits at once, and optimizing performance alone yields skills that pass benchmarks but degrade on real deployment infrastructure. MOCHA navigates the constraint frontier with *Chebyshev annealing*. It operationalizes the complexity penalty $\Omega(z)$ of the gate, and it shows that the scalar Ω of Section III simplifies what is, in production, a Pareto frontier.

Ratchet [36] places the bottleneck elsewhere. LLM-authored skills deliver +0.0 pp over no-skill baselines while human-curated ones deliver +16.2 pp, which points away from the optimizer, the mechanism that proposes and accepts edits, and toward *lifecycle management*: whether skills are written, retrieved, curated, and retired correctly. Four hygiene mechanisms (outcome-driven retirement, a bounded active cap, meta-skill authoring, and pattern canonicalization) lift the +0.0 pp floor substantially before any optimizer runs.

The finding clarifies a precondition of descent rather than refuting it. The gate needs a meaningful signal: if retrieval injects the wrong skill into every rollout, the proposal process conditions on corrupted evidence and descent degrades to random search. Ratchet's mechanisms keep residual capture and mode discovery (operations 1 and 2) working. Retirement keeps stale artifacts out of the residual signal, the active cap keeps scaffold bloat from making retrieval unreliable, and canonicalization collapses near-duplicates so that proposals condition on one clean artifact set. A system with strong hygiene and no optimizer sits at Stage 1 with a clean scaffold; one with a strong optimizer and poor hygiene sits at Stage 2 with a noisy loss. Stage 3 needs both.

SkillsVote [37] carries the lifecycle argument to an open ecosystem. It profiles a million-scale open-source skill corpus for environment requirements, quality, and verifiability, synthesizes tasks for the verifiable subset, and exposes skills through agentic library search before execution. After execution it splits each trajectory into skill-linked subtasks and attributes their outcomes to skill guidance, agent exploration, environment, or result signals; only successful, reusable discoveries pass into evidence-gated updates. Attribution thus happens at the admission gate rather than inside the optimizer, credit assignment used as a filter on residual capture. The redundancy and environment sensitivity SkillsVote measures are the bloat and drift of Section X, arising here from many authors rather than one loop.

S5.8 Cross-task transfer and evidence radius

Several papers measure what Section III calls the *evidence radius* of a scaffold artifact, how far an edit validated in one context keeps helping when redeployed elsewhere, without using the term.

In-context Training [ICT; [38]] tests cross-task self-improvement: whether experience distilled from one set of tasks transfers to unseen ones. Its finding that "current reflection-based methods can only self-correct within a single task instance" measures the radius at its lower bound, one task. ICT tries to widen it by distilling reflections into reusable lessons, which instruments Loop 2 empirically: does an edit that passed the gate on one task's L_D still help across a distribution of unseen tasks?

Combee [39] reaches the same measurement through scale. ACE [8] showed that a system prompt which accumulates and organizes strategies through agentic interaction improves on a static prompt for a single agent; Combee learns from the traces of many agents running in parallel. Each proposal then draws on a wider deployment distribution, and the promoted artifact is tested against a larger average of patch losses. This is not cross-organizational promotion. Combee measures how much of the evidence-radius benefit parallelism inside one organization captures.

CORAL [15] runs long-lived agents that explore, reflect, and collaborate through shared persistent memory, asynchronous execution, and heartbeat-based interventions. A finding one agent writes to the shared store becomes available to all, so the store is the cross-agent promoted artifact and the heartbeat a form of auto-gated Loop-2 promotion; there is no versioned lineage, RBAC, or rollback. CORAL is the only fully asynchronous multi-agent Loop-2 promotion mechanism among the systems classified before the second pass; in the full corpus, Evo2Team and ShareMem add cross-team and cross-user promotion without versioning (Section S6.3).

PACE [16] is a production-oriented two-timescale design for small language-model agents. The fast timescale adapts prompts, parsers, validators, and control logic from failure signals within a deployment, as Loop 1 does; the slow timescale consolidates across the deployment's own episodes, a Loop 2 whose radius is one

deployment. The two-loop structure holds on small, frozen production models, not only under frontier-model assumptions, and persists even when the optimizer and the target are the same frozen small model.

SkillClaw [19] and **FederatedSkill** [18] are the two systems of these walkthroughs whose Loop 2 crosses users rather than tasks or agents; in the full corpus, EvolveNet, ShareMem, and XRepoSkill also promote across users, clients, or repositories, each inside one operator (Section VII). SkillClaw aggregates many users' trajectories, turns recurring behavioral patterns into skill-set updates, and synchronizes the updated skills across users; its evaluation runs a day-night cycle in which simulated users generate trajectories by day and the system reflects, proposes, validates, and deploys by night. FederatedSkill exchanges semantic skill diffs, structured patches over a local library shipped with the task reward that produced them, so that clients share what they learned without sharing raw trajectories. A server-side evolution agent aggregates the diffs into a per-client capability matrix (covered, absorbing, broken, gap) and applies, adapts, or rejects each peer patch per client, logging source patch, reward, and justification for every modified path. Across 20 task families it reports up to +44.4% success and -37.5% compute against isolated self-evolution. FederatedSkill is the first surveyed system in which Loop 2 aggregates artifact updates rather than promoting a single artifact, and its decision log is the first lineage record at an aggregator. Its acceptance, though, uses the source client's reward rather than a held-out evaluation at the receiving client.

S5.9 Synthesis: what the convergence establishes

Section VIII draws four conclusions from the families, and the walkthroughs support them as follows.

1. **The scaffold-versus-weights decomposition is independently confirmed.** GEPA, Meta-Harness, MOSS, RCL, and PACE all begin from the observation that model weights and the external artifact layer are distinct adaptation surfaces with different cost, reversibility, and scope properties. No surveyed paper disputes this decomposition; the variation is in which substrate each paper targets (skill document, harness code, system prompt, shared memory).
2. **Credit assignment is the central engineering bottleneck.** AHE, CANTANTE, Meta-Harness, the harness-optimizer evaluation of Ong et al. [33], HARNESSEVO, and SkillsVote all identify credit assignment, attributing a system-level failure to a specific scaffold coordinate, as the problem that separates disciplined Loop-1 evolution from trial-and-error. It is operation 2 of Section S3.1, the attribution that the correspondence note of the formal appendix requires, and it remains unsolved in the general multi-substrate case.
3. **Lifecycle hygiene is a precondition for descent, not an alternative to it.** Ratchet establishes that the gradient-analog mechanism of Section III requires a maintained scaffold. A dirty artifact pool (stale, bloated, miscurated) keeps the gate G_t from receiving a meaningful signal. Hygiene and optimization are sequential dependencies, not competing paradigms.
4. **No surveyed system combines Loop-1 automation with cross-organizational Loop-2 governance.** SkillOpt has the strongest Loop-1 closure and the most systematic evidence-radius measurement, but it does not implement cross-organizational promotion, versioned lineage with RBAC, or rollback. CORAL implements asynchronous cross-agent promotion within one system but not across organizations. PACE implements two-timescale consolidation within one deployment. Meta-Harness implements Loop-1 closure over the harness substrate but no Loop-2 promotion mechanism. FederatedSkill federates skill patches across clients under one operator, with a decision log but no RBAC or rollback, and EvolveNet adds a held-out merge gate and round rollback without lineage or RBAC. The four-property gap of Section IX is narrowed on (b) and (d) by different systems and closed on none, and the independent convergence of several research threads on the same single-organization ceiling confirms it.

S5.10 SkillOpt against Section IX's four properties

Mapped onto Section IX's four properties of governed cross-tenant scaffold optimization:

- (a) **Auto-gated Loop 1.** *Satisfied.* The held-out selection-split gate at strictly-greater margin instantiates G_t.

- (b) **Multi-tenant cross-org promotion.** *Absent.* SkillOpt is a single-organization research system; there is no governed cross-tenant aggregator.
- (c) **Versioned lineage with RBAC.** *Partial.* The exported `best_skill.md` is a versioned artifact, and the protected slow-update field separates fast and slow consolidation. There is no documented RBAC layer or audit-trail policy, and `edit_apply_report.json` is an operational log rather than an access-controlled lineage system.
- (d) **Rollback.** *Absent.* The training loop selects the best validation-gated skill and exports it; no rollback envelope on deployed artifacts is reported.

The cross-model, cross-harness, and cross-benchmark transfer experiments reported by SkillOpt are *empirical evidence-radius measurements*: a skill optimized in one (model, benchmark, harness) cell continues to improve performance when redeployed in nearby cells without further optimization. In Section III vocabulary, this is a measured Loop-2 *radius* for the artifact, not a *governance mechanism* for promoting it. Both readings are useful: SkillOpt is the strongest Loop-1 proof and supplies the first systematic transfer evidence at the scaffold layer, while the Section IX cross-tenant governance triad ((b), (c), (d)) remains unfilled.

S5.11 Cost and bloat

SkillOpt reports 0.6M to 46.4M training tokens per absolute test-point gain (their Table 6), with the procedural benchmarks (SpreadsheetBench, OfficeQA, LiveMath) at the cheap end and search/visual benchmarks (SearchQA, DocVQA) an order of magnitude more expensive per point. The deployed artifact remains 379–1,995 tokens regardless. This is, to our knowledge, the first published cost-per-point figure for scaffold-layer descent, and it complements the Section X scaffold-bloat discussion (and the Ratchet hygiene-precondition finding walked in Section S5.7): the textual learning rate η_t with cosine decay keeps the deployed skill compact even when the optimizer explores many more edits than are accepted (only 1–4 edits survive into the final skill across the six benchmarks). The mechanism that prevents the deployed artifact from bloating is exactly the complexity penalty $\Omega(z)$ of Section III, instantiated as a hard cap on accepted edits per step rather than a soft regularizer.

S6. Audit of the first-pass draft's absolute claims against the full corpus

The first-pass draft of this survey, written on the 130 systems of the first pass, made absolute claims of the form *no surveyed system*, *only X*, *first*, *absent*, and *empty cell*; each row below sets one of them against the full corpus of 256 systems and records what that corpus shows. A claim **holds** when the full corpus bears it out as worded, is **sharpened** when it is true and the corpus supports a more precise statement, is **revised** when the corpus contradicts its wording, and is **dropped** when the article no longer makes it; for sharpened and revised claims the Evidence column also states the claim as the full corpus supports it. Evidence names the systems that decide each verdict, and parenthetical numbers such as 2608.04968 are arXiv identifiers.

S6.1 The four properties of Section IX

Claim	Verdict	Evidence
No public system combines (a) auto-gated Loop 1, (b) cross-organization promotion, (c) versioned lineage with role-based access control (RBAC), (d) rollback	**holds**	Across the 126 second-pass systems: no system demonstrates promotion across organizations, and EvolveNet has a partial (b) inside one operator (its clients are benchmark partitions, "organizations" only in motivation), while RASO, which retrieves from a public corpus, promotes nothing; MemClaw is single-tenant in its evaluation, and Buildrix is an open community hub with no gate; (c) appears as lineage without RBAC everywhere (EDDOps, Agentic Configuration Management (ACM), MemClaw, Letta, Mem0 history) and as RBAC without versioned promotion in the MCP control planes (agentic-community gateway, TrueFoundry); (d) is demonstrated by Governed Capability Evolution and EDDOps and claimed by EvolveNet (round rollback) and EvoUndo (recoverability as admission), none of which has (a)+(b).
"No surveyed system meets the FedAvg limit; FederatedSkill comes closest"	**sharpened**	EvolveNet (2608.04968) comes nearer: it broadcasts a shared harness, optimizes it locally at each client, merges on the server only when the merge fixes at least as many held-out items as it breaks, and rolls back to the previous shared harness after a failed round. It is still selection and merge over program edits, not averaging, so neither it nor FederatedSkill meets the FedAvg limit.
Fourth conclusion on the research families: "FederatedSkill comes closest … narrowed on one property, closed on none"	**revised**	The two closest approaches are FederatedSkill (decision log; no gate at the receiving client) and EvolveNet (held-out merge gate and round rollback; no lineage or RBAC). The gap is narrowed on (b) and (d) by different systems and closed on none (Table IX-1).
"no member of the Section VIII research families provides (d), and only FederatedSkill provides even a partial (b)"	**revised**	(d) appears in EvolveNet (round rollback), SkillSmith and SkillAudit (edit-level rollback inside one agent), Growing Harness, WDA, and Co-Harness (rollback within an optimization run; Co-Harness adds a partial (c), a versioned diff registry without access control), and EvoUndo (recoverability as an admission test); EvolveNet provides a partial (b). Rollback appears inside single deployments and inside one federated round; no system rolls back a promotion that has crossed an organization boundary, because none makes such a promotion.

S6.2 Matrix cells

Claim	Verdict	Evidence
"Three cells are populated; the rest are empty for structural reasons"	**revised**	Single-loop and governance-only systems occupy these cells, under the definition of Section III that a gate records who decides and not who supplies the candidate: [None × Human] (MCP registries: Azure API Center, agentic-community MCP Gateway & Registry, TrueFoundry; Buildrix; the Anthropic skills repository; ACM), [Human × None] (VerificAgent, SkillChain's second-stage reflection gate, Davis et al. governance conversion), [Auto × None] (Letta's ungated push; Mem0 Platform, TOKI, Learning on the Job, RASO), [None × Auto] (Governed Capability Evolution; EDDOps, with a sign-off after the threshold), [None × None] (Official MCP Registry; Enoch's migration protocol; Continual Harness's unchecked self-edit; the revocation guard). Two cells hold every audit passer whose promotion has a gate, a third holds the governance-first platforms, Letta passes with no promotion gate, and the human-gated row holds only curation (Section VII).
"[Human × Auto] is empty by engineering logic . . . no published example exists"	**holds**	No system in the full corpus puts a person on the local gate of a failure-driven loop and promotes by program. MCP Gateway & Registry adds an automated security scan to a human admission, but it has no failure-driven local loop, so it sits in [None × Human].
"[Human × Human] is empty in the surveyed corpus"	**holds, after re-coding**	The registries and the community hub that a first reading places here have no failure-driven loop and sit in [None × Human] (first row). The occupant the first-pass draft predicted, solo or small-team workflows, is documented by "Who Maintains Agent Skills?" (2609.05677): across five public skill repositories every substantive edit is human-authored and human-merged, 62% are AI co-authored, and there is 1 deprecation in 254 edits; that shape is [None × Human] too, because no loop fires from session signal (Section VII).
"[Human × None] is structurally improbable"	**revised**	VerificAgent (agent-drafted memories, human fact-check, frozen bank), SkillChain's second-stage human reflection gate, and the Davis et al. governance conversion occupy the cell. It is the human-curation cell, single-loop by construction.
"Nearly every auto-gated system fires at n = 1" (accepts an edit on a single evaluation)	**revised**	Batched or statistical acceptance is common, and has been since mid-2026: SAGE-gate (paired per-item test), Future AGI (paired bootstrap CI), AutoSaddler (mini-batch failure signals), MGM, ModularRSI (evidence across tasks), VaG (3-of-3 critics), EvolveNet (fixed $\geq$ broken on held-out), VACE, HackProbe, PROCTOR. Evidence is batched before acceptance, with SAGE-gate and VaG as instances (Section VII).

Claim	Verdict	Evidence
"no surveyed system has formalized agreement as a k-of-n acceptance rule"	**revised**	VaG requires all three heterogeneous critics to pass (3-of-3), and SkillSpec is consensus-gated, so agreement-based acceptance exists. The distinction between agreement and averaging stands: under agreement the false-accept rate decays as p^k, under averaging the variance decays as $1/k$ (Appendix XIII).
"Only AlphaEvolve demonstrates explicit multi-layer depth"	**sharpened**	HarnessBank, JIT-Agent, and MGM keep archives for lineage, and VaG adds tiered subset selection; none pairs a cascade evaluator with a diversity archive. AlphaEvolve is the only system that combines the two.

S6.3 "Only" and "first"

Claim	Verdict	Evidence
CORAL "the only asynchronous multi-agent Loop-2 mechanism in the corpus"	**revised**	Evo2Team (2609.34135) promotes routing and communication skills across teams of 4–32 agents with target-side confirmation, and ShareMem pools memory across users. CORAL is the only such mechanism among the systems classified before the second pass; in the full corpus Evo2Team and ShareMem add cross-team and cross-user promotion without versioning (Section S5.8).
AlphaEvolve Strassen claim	**holds** (attributed)	The claim is AlphaEvolve's reported improvement on Strassen's algorithm for 4×4 matrix multiplication. The result is taken from the AlphaEvolve report and was not independently measured; with that attribution in the same sentence, the claim holds.
"CASCADE reports the largest within-benchmark ablation gain in the corpus"	**dropped**	Not established for the 126 second-pass systems, and ablation gains measured on different benchmarks are not comparable. The article does not make the claim.
ChronoMem and rollback repair "the first surveyed instances of the rollback envelope"	**revised**	Memory rollback also appears in Mi-Memory (as a design) and Letta (git history), and checkpoint-rollback studies (2608.29381, 2608.15939) show rollback unsafe. VerMem keeps prior versions and soft-deleted records but has no operation that reverses an edit; the "Restore" in its tool table moves episodic fragments of the current task back into context. ChronoMem and rollback repair are among the first instances, and the checkpoint-rollback studies are the counter-evidence.
Cuadros "only surveyed memory system with full provenance + versioning + correction"	**holds**	The memory-governance systems of the second pass (TOKI, PPMF, TMA-NM, MemClaw, He & Yu) each carry provenance or lineage without correction loops or versioning, and the auto-gated memory loops (Recuris, MemGuard, VerMem, Compiled Memory) carry no lineage.

Claim	Verdict	Evidence
SkillOpt “cleanest published instance” / “first to instantiate all four simultaneously”	**sharpened**	SkillOpt-Lite (2607.03451) removes two of the four controls and matches or beats SkillOpt; VALVE instantiates a held-out gate that rejects a candidate before commit and never reverts an accepted update, and SkillSmith and SkillAudit instantiate held-out gates with edit-level rollback. SkillOpt is the first system we found that instantiates all four, and its own follow-up shows that two of them are stabilizers rather than necessities (Section S5.3).
FederatedSkill “first … federated aggregation … first lineage record at the aggregator”	**holds**	EvolveNet (August 2026) postdates it and keeps no lineage record.
SkillOpt “first published cost-per-point figure”	**holds**	The full corpus contains no earlier figure.

S6.4 Absences

Claim	Verdict	Evidence
“production deployments sit at Stage 1 or 2; none reaches Stage 3”	**sharpened**	Spotify (2609.30297) runs automated prompt and planning optimization gated by human launch and A/B metrics; this is Stage 3 on Loop 1 with a human Loop 2: production deployments reach Stage 3 on the local loop only behind a human promotion gate (Section III). SkillForge (2604.08618) stops short of it: a judge that agrees with experts more than nine times in ten flags the failed replies that drive each skill rewrite, but the rewrite is committed without an acceptance test, and support engineers review replies, not skill versions.
No S1 system scores 5; “no instruction system closes both loops”	**holds**	On S1, n=15 and the maximum score is 4. Adoption studies point the same way: context files raise cost by more than 20% without raising success (Gloaguen), and context files dominate configuration while skills are rare (Galster).
“Anthropic’s skills repository is one-pool with PR review but no automated eval gate”	**holds**	The study of public skill repositories (2609.05677) observes human merges and no evaluation gate.
“correctable, versioned memory is rare; production memory lacks failure-triggered correction and versioned lineage”	**revised**	Letta Context Repositories (git-backed memory, every edit committed, shared memory auto-pushed), Mem0’s per-memory history and feedback endpoint, and Zep’s bitemporal invalidation make lineage common in production memory. Failure-triggered correction is not common, and 2609.08258 shows revoked facts still outranking their replacements in five backends. Production memory is versioned, but correction is not enforced (Section VI).

Claim	Verdict	Evidence
No S5 system scores 5; "topology evolution is rare"	**sharpened**	With MAE and Memento-Skills corrected to 4 (Section S6.7), S5 has no score of 5 in either pass. Gated topology evolution exists: TOCOMAS keeps only candidates that satisfy structural invariants; Evo2Team promotes across teams; AgentGrad and HarnessForge update S5 under an evaluation gate. OptiMAS updates S5 through an LLM optimizer that reads a running validation score and reverts an edit only when its own analysis refutes it, with no keep-if-better test (its Algorithm 2 returns the last configuration, and the only programmatic check is structural validity). Topology evolution is rare and, where it exists, gated by evaluation alone except in TOCOMAS, which adds a structural invariant, and OptiMAS, where the evaluation informs the edit rather than gating it.
"no system matures all six at once"	**holds**	None of the 126 second-pass systems scores 5 on any substrate.
"Failure-triggered memory is rare in production"	**holds**	One field report exists (SIx Harness, N=1, self-reported); Letta's dreaming writes from conversation history, not from failures.
"Cross-customer skill promotion is nearly absent"	**holds**	ShareMem, EvolveNet, and XRepoSkill promote across users, clients, or repositories inside one operator; none promotes across customers or organizations with evidence.
"Memory versioning is mostly unsolved outside Cuadros et al. and ChronoMem"	**revised**	The evidence is the same as for the claim that production memory lacks versioned lineage (fourth row of this table): Letta, Mem0 history, Zep bitemporal facts, TOKI audit rows, and MemClaw supersession chains all version memory. Memory versioning is common, not confined to Cuadros et al. and ChronoMem.
"Scaffold-level curriculum is unspecified in nearly every surveyed system"	**sharpened**	Task-CoEvolve (variance-weighted task sampling) and AutoSaddler (mini-batch failure signals) specify it; they remain the exception.
"Held-out evals at Loop-2 promotion are absent"	**revised**	Receiving-side evaluation exists: ShareMem (evaluation tasks belong to a different user), XRepoSkill (independent verification in $\geq$2 repositories), Evo2Team (target-side confirmation; 4 of 22 transfers still fail on new graphs). VACE's held-out gate compares two harnesses on the next checkpoint of the same training run, so it is a Loop-1 check rather than receiving-side evaluation and is not counted here. EvolveNet gates the merge on a server-side held-out slice but runs no check at the receiving client, and Federated Agent Optimization (2610.01195) proposes a receiving-client gate without implementing it. Receiving-side evaluation at promotion is rare, not absent (Section VI).
Research full-auto cluster: "enterprise governance is uniformly absent"	**holds**	No second-pass [Auto $\times$ Auto] system reports RBAC or audited promotion.
Governance-first cluster: "no patch-local inner loop"	**holds**	EDDOps, ACM, Governed Capability Evolution, and the MCP control planes all lack an inner loop.

Claim	Verdict	Evidence
"AGENTS.md in over 60,000 repositories"	**revised**	Adoption is measured directly rather than by one repository count: a 2,853-repository study (Galster), a 128,018-project adoption study (Robbes: 22–29% of coding-agent projects), and 36,710 repositories mined for loop engineering (Lulla, 2608.21884), alongside a controlled study in which context files raise cost without raising success (Gloaguen; Section VI).
"no surveyed system treats pruning or expiration as a first-class gated operation (outside Ratchet, SkillOpt, ChronoMem)"	**revised**	Retirement is a gated operation in at least eleven skill systems (Library Drift: retire when n $\geq$ 100 and contribution $\leq$ -0.10 with a cap of 50; SkillSmith; SkillOps; ASSAY; HiSME; SkeMex; MUSE-Autoskill; SkillMAS; SafeEvolve; Double Ratchet; SkillGen), and expiry is a product control in Mem0, Zep, and LangGraph Platform. Pruning and expiry are first-class operations; what remains rare is pruning of a promoted artifact's descendants, which VaG shows is structurally irreversible.
The failure surface is "largely unmitigated in production"	**holds**	Revocation is unenforced in five production backends (2609.08258), checkpoint rollback is unsafe in three frameworks (2608.29381), and 16 deployed frameworks keep no evidentiary record (Hearsay, 2609.32495).
Multi-sample agreement is "under-explored; SkillOpt-Lite nearest"	**revised**	The evidence is the same as for the two acceptance-rule claims of Section S6.2, single-evaluation acceptance and k-of-n agreement: SAGE-gate, VaG, and SkillSpec. Agreement-based acceptance is instantiated, not under-explored (Appendix XIII).
"No surveyed paper disputes the scaffold-vs-weights decomposition"	**holds**	SIA (2605.27276) updates harness and weights together but keeps them as distinct levers.
Credit assignment "remains unsolved in the general multi-substrate case"	**holds**	HARNESSEVO (slot attribution), AgentGrad (intervention-based), and CANTANTE (contrastive) each solve one substrate's case.

S6.5 The three predictions of Section IV

Section IV predicts from the signal classes which cells of the design space each class can reach, and Section VII confirms the predictions; the rows give the systems behind each verdict, in the order Section IV states them.

Claim	Statement	Verdict	Evidence
First	An [Auto × Auto] system whose gate estimates the loss change per edit should run on a C1 signal or a simulated C2 environment; one that works on a C4 signal should prune proposals rather than estimate	**holds**	Before the second pass: DGM, Voyager, AlphaEvolve, SkillOpt, SAGE, EvolveR, CoEvoSkills (C1 or simulated C2); NanoResearch (semantic merge, C4) and CORAL (heartbeat, C4) prune. AutoManual estimates nothing: its Builder reads each episode's trajectory and binary outcome and writes or rewrites rules by its own judgment, with no test of the edit, and its Consolidator merges and deletes rules once they exceed twelve, so it decides by a model judge over a simulated outcome (C3 over C2) and prunes, and the claim does not bear on it. SkillRL and AutoAgent no longer sit in [Auto × Auto] (Section S6.7). Second pass: EvolveNet, Evo2Team, XRepoSkill (C1 code or benchmark signals); ShareMem (judged consolidation, C3, measured on Mind2Web); Letta (no gate).
Second	[Auto × Human] should hold only C2 and C3 systems, because the human on Loop 2 substitutes for the exact signal that Loop 1 lacks	**holds**	CASCADE (C3), Sierra (C2/C3), Devin (C2 merge outcome), Spotify (C2 A/B behind human launch). No C1 system occupies the cell; SkillForge, also C3, promotes nothing and sits in [Auto × None].
Third	No scaffold system should yet calibrate a judged gate under the reflexive condition	**sharpened**	Three scaffold systems report calibration: SkillForge (>90% agreement), SkillChain (κ 0.72 to 0.91), and SkillCoach (trajectory-filter consistency 82→96% against human gold); none calibrates a judge that is itself being edited. On the weight side, no method that co-evolves a rubric or judge reports agreement with people (Section S6.6).

S6.6 The rubric-reward comparison

Reinforcement learning from verifiable rewards trains weights on exact signals, and extending it to writing and other open-ended tasks has produced the judged gates that a C3 scaffold loop can borrow. These are weight-side methods and the corpus does not score them; Section IV reads one distinction from them, whether the rubric or judge is held fixed while the policy learns or moves with it. The comparison covers fourteen papers, thirteen methods and one survey. Of the methods, eight hold the rubric or judge fixed during policy optimization and five co-evolve it. Five report a human-agreement figure. Four of them hold the gate fixed, and the fifth, RLPR, co-evolves a probability reward rather than a rubric, checked against human labels. No method that co-evolves a rubric or judge reports one.

Gate during policy optimization	Papers	Members	Agreement with human judgment
Held fixed	8 methods	Rubrics as Rewards, an instance-specific checklist scored by a fixed judge [40]; Writing-Zero, a pairwise judge trained on writing principles and frozen for policy training [41]; reference-guided evaluation [42]; an essay scorer trained against a fixed rubric of binary items [43]; a study of cheap training-time verifiers [44]; Rubric-Grounded RL, the Open Rubric System, and query-specific rubrics learned from human preferences [45–47]	Reference-guided evaluation reaches 79.1% agreement with human annotations; the essay scorer reaches a quadratic weighted kappa of 0.803 against a single human rater's 0.805; the cheap-verifier study calibrates against language-model golden judges, not people
Co-evolved with the policy	5 methods	RLPR, scoring by the policy's own probability of a fixed reference [48]; Rubric-ARM, alternating updates of a rubric generator and a judge at the reward-model stage [49]; EvoRubrics, SCORE, and RubricEM, updating rubrics adversarially or from a rubric buffer within training [50–52]	RLPR only, for its probability reward against human labels; no method that co-evolves a rubric or judge reports a figure
Survey	1 survey	A survey of rubric evaluation, which names the dynamic regime as the trend [53]	—

The only co-evolving method with a human-agreement figure, RLPR, co-evolves a probability reward, not a rubric. No method that co-evolves a rubric or judge reports agreement with people. That is the reflexive-gate problem of Section IV, seen from the weight side.

S6.7 Recount of the full corpus

The full corpus holds 256 unique systems, 130 from the first pass and 126 from the second; 204 form the core corpus and 52 the contrast corpus. None of the 126 second-pass systems scores 5 on any substrate. The composite two-loop audit passes 11 research systems and 3 industry exemplars: Sierra Agent OS 2.0, Cognition Devin, and Letta's Context Repositories, whose shared-memory push carries git lineage but no gate. The 6 second-pass systems that promote across contexts with no versioning or lineage on the promoted artifact are EvolveNet, Evo2Team, XRepoSkill, ShareMem, Spotify, and Future AGI. Of these, EvolveNet alone carries rollback and so enters under the governance relaxation of Section S2; the first five enter only if the audit drops its governance requirement altogether; Future AGI's evidence is T5 only, so it enters not at all.

Two first-pass rows are corrected before the recount. The row "Governed Collaborative Memory / SSGM" conflated Cuadros et al. (2605.04264) with SSGM (Lam et al., 2603.11768); it is split, and SSGM, a conceptual framework, scores 2. MAE and Memento-Skills were scored 5 on S5 in the released CSV while the first-pass draft's table said no orchestration system reaches 5; both close one loop only and score 4 under the uniform rubric, which moves the first-pass S5 mean from 2.68 to 2.58.

Nine second-pass rows had their gate labels re-coded on 2026-10-08 under the definition of Section III, which records who decides and not who supplies the candidate: Letta's Loop-2 label moved from Auto to None (an ungated git push of shared memory; the row still passes the composite audit on its lineage); the Loop-1 label of Azure API Center, the agentic-community MCP Gateway & Registry, the Official MCP Registry, TrueFoundry, Buildrix, the revocation guard, and "Who Maintains Agent Skills?" moved from Human to None (artifacts registered or authored by people, with no failure-driven loop), and Enoch's checkpoint-and-validate

protocol moved to None on both loops (a migration of one agent's state, not a promotion). The MCP Gateway's Loop-2 label now records its automated admission scan beside the human approval, and the field study's Loop-2 label records that maintainers merge. No score changed; the cells of Section VII and Section S4.2.1 follow the re-coded labels.

Five more rows were corrected on 2026-10-08 after a reading of the full texts against the same definition. RASO's Loop-2 label moved from Auto to None: it accepts a rewritten skill only if the rewrite beats the incumbent on a validation split, but the optimized skill never leaves the task, and its use of a public corpus is inbound retrieval, not a promotion. Three first-pass systems leave the composite audit. SkillForge moves from [Auto × Human] to [Auto × None]: its judge flags the failed replies that drive each skill rewrite, the rewrite is committed without an acceptance test, its support engineers review replies and not skill versions, and no skill version leaves the scenario deployment. SkillRL's Loop-1 label is None: a per-category success rate below 0.4 triggers new skill proposals, and the skill bank takes each one as an unconditional union, a step that runs unchecked. AutoAgent moves from [Auto × Auto] to [Auto × None]: its analyzer accepts text revisions on an uncalibrated consistency judgment, and each revision stays, unversioned, in one agent's own store. Co-Harness's governance entry now records a partial (c), its versioned diff registry, and (d) within one run, and its signal class records C1 for the held-out gate beside C2 for attribution. The audit therefore passes 11 research systems rather than 14, [Auto × Auto] holds ten research systems rather than twelve, the [Auto × Human] cell of Section S4.2.1 holds CASCADE, Sierra Agent OS 2.0, and Cognition Devin, and six systems rather than seven promote without lineage. No score changed.

The recount runs over all substrate-level rows and gives the counts of Table V-2; its last line is the first pass alone (systems / core / contrast) after the two corrections.

```
rows: 269  unique systems: 256  core: 204  contrast: 52
substrate            n   mean  score-5 systems
INTEGRATED           52   3.81  AutoAgent, NanoResearch, SkillRL
S1_Instructions      15   3.07  (none)
S2_Skills            50   3.34  COSPLAY, SAGE
S3_Memory            50   3.06  Governed Collaborative Memory (Cuadros et al.)
S4_Tools             20   3.65  Harvey, Hippocratic AI, MCP
S5_Orchestration     37   3.08  (none)
S6_Governance        45   3.02  AGENTS.md / AAIF, Braintrust, LangSmith Hub, Vellum
First pass alone (after corrections): 130 / 99 / 31; S5 mean 2.58.
```

S7. Extended related work

The main article makes four design commitments that another survey could make differently: it classifies systems by whether a feedback loop closes over a persisted artifact, rather than by which mechanism is edited; it labels the gate on each of two loops as algorithmic, human, or absent; it classifies the signal that feeds the gate by verifiability; and it audits governance against four named properties. Each commitment meets a separate line of prior work. This section traces what each line established, the setting it assumed, and the condition that changes here, so that a reader can see the differences without the article having to argue for them. Every description below is taken from the cited work's own abstract, method section, or documentation; the evidence record is released with the article.

S7.1 Loop closure as the unit of classification

The surveys of self-evolving agents organize the field by what is evolved and when. Gao et al. [54] ask what, when, how, and where to evolve, with a timing axis that separates adaptation inside a test-time episode from adaptation between episodes, and they note that a human in the loop can validate a newly created tool. Fang et al. [55] give a unified frame of system inputs, agent system, environment, and optimizers, and cover single-agent, multi-agent, and domain-specific settings. Zhou et al. [56] taxonomize 108 self-evolving coding methods by evolution target, timing, and code-specific signal, and they come closest to a gate: their admission rule accepts an update when it improves a metric by a margin and passes a check, and they observe that a

passing test "checks only selected behavior". Ren et al. [57] describe an agent as a foundation model times an operational scaffold and treat self-improvement as a self-induced update operator organized by target and driving signal. Surveys of agent memory [58–60], context engineering [61, 62], and agent skills [63–65] do the same for their substrate: each catalogues mechanisms and, in the skills surveys, lifecycle stages.

What this line established is the inventory. The substrates this article scores are the ones these surveys name, and the mechanisms it finds in the corpus are the ones they catalogue. The setting they assume is a mechanism-centered one: a system is described by what it edits. The condition that changes here is the question asked of each system. Whether a loop closes at all, whether it closes under a gate, and whether a second loop promotes the result are properties that the mechanism inventory does not determine, and they are what the two-loop audit of Section V measures. Zhou et al.'s admission rule is the nearest precedent for the gate; it is stated for one domain, code, where the check is a test, and this article asks what the rule becomes when the check is a judge or a delayed outcome.

S7.2 The gate as algorithmic, human, or absent

Three lines bear on gate type. The first is prompt and program optimization, which a reader will expect to see named. APE treats the instruction as a program and selects among LLM-proposed candidates [66]; ProTeGi forms natural-language "gradients" from minibatches that criticize the current prompt and searches edits with beam search [67]; OPRO has the language model propose new instructions from a prompt that lists previous candidates with their scores and reports gains of up to 8% on GSM8K and up to 50% on Big-Bench Hard over human-designed prompts [68]; EvoPrompt applies evolutionary operators to a prompt population and improves it against a development set, with gains of up to 25% on BBH across 31 datasets [69]; Promptbreeder evolves task prompts and the mutation prompts that modify them, evaluated on arithmetic and commonsense reasoning and on hate-speech classification [70]. DSPy compiles pipelines of language-model calls into text-transformation graphs and optimizes each pipeline to a metric by creating and collecting demonstrations [71], MIPRO proposes instructions and demonstrations jointly for multi-stage programs under a stochastic mini-batch surrogate [72], TextGrad backpropagates textual feedback through the variables of a compound system [4], and Trace generalizes the pattern to any computation whose execution trace can be shown to an optimizer [73]. This line established that a development-set score is a usable gate for an algorithmic proposer and that the proposer can be the model itself. The setting it assumes is a single optimization run against a fixed task distribution. The condition that changes here is persistence and scope: these optimizers produce a prompt or program as the optimizer's output, not a versioned deployment artifact edited from session signal and promoted under a second gate. That is why OPRO and TextGrad score below the core-corpus threshold and DSPy, which persists only optimizer state, closes no second loop; Section S4.1 lists all three. SkillOpt is the point where the lineage crosses into the corpus, by adding a persistent skill document, a held-out acceptance gate, and a rejected-edit buffer to the same proposer (Section VIII).

The second line optimizes the scaffold as a whole rather than one prompt, and it is the nearest precedent for the loop of Section III. Agent symbolic learning treats prompts, tools, and the pipeline that connects them as symbolic parameters and updates them with textual losses and gradients [74]; STOP has a scaffolding program recursively rewrite the code that improves it [75]; a self-improving coding agent edits its own harness and raises its benchmark score as a result [76]; and workflow search optimizes the orchestration graph itself, by node and edge optimization over agent graphs [77] or by tree search over code-represented workflows [78]. These established that the whole scaffold is an optimizable object and that textual feedback can drive its updates. Each runs within one deployment under a benchmark score, with no promotion to other deployments and no governance of the artifact it produces; the two-loop audit of Section V is the measurement of exactly those two absences.

The third line is the classic single-loop agents and their memories: Reflexion's verbal reinforcement into episodic memory [7], the reflection-and-importance memory of generative agents [79], Voyager's growing skill library [80], the experience bank ExpeL extracts from training tasks [81], MemGPT's virtual context management [82], Agent Workflow Memory's induced and reused workflows [83], the Zettelkasten-style organization of A-MEM [84], the persistent test-time memory of Dynamic Cheatsheet [85], the evolving context playbook of agentic context engineering [8], and the in-context iteration of Self-Refine and ReAct [2, 3]. They established that a frozen model can improve across episodes through an artifact it writes, and the

cognitive-architecture framing of Sumers et al. [86] gave that artifact the episodic, semantic, and procedural division this article's memory substrate uses. Reflexion, generative agents, Voyager, ExpeL, MemGPT, Agent Workflow Memory, A-MEM, Dynamic Cheatsheet, and the context playbook persist the artifact and close Loop 1; Self-Refine and ReAct do not persist it. None promotes the artifact beyond the agent that wrote it, which is the absence the second loop of this article is defined to measure.

S7.3 Verifiability of the feedback signal

No surveyed work classifies systems by the verifiability of the signal that feeds the gate, but three lines come near it. Guo et al. [87] note that software engineering provides mechanical feedback, in the form of builds, unit tests, linters, and execution logs, that most other domains lack; Zhou et al. [56] note that even a passing test is incomplete feedback. The judged end of the spectrum has its own measurements: language-model judges agree with human preference at rates that make them usable evaluators of open-ended answers [88], a proxy reward optimized past a point lowers the true score [89], and people who grade model outputs revise their own criteria as they grade, so that the evaluator drifts along with the evaluated [90]. The second line is the weight-side effort to extend reinforcement learning from verifiable rewards into domains without verifiers. Rubrics as Rewards generates an instance-specific checklist before training and scores responses with a fixed judge [40]; Writing-Zero trains a pairwise generative judge on writing principles and holds it fixed during policy training [41]; reference-guided evaluation reaches 79.1% agreement with human annotations across the datasets its authors test [42]; an essay scorer trained against a fixed rubric reaches a quadratic weighted kappa of 0.803 against a single human rater's 0.805 [43]. A second group moves the gate with the policy: RLPR scores by the policy's own probability of a fixed reference [48], Rubric-ARM alternates updates of a rubric generator and a judge [49], and EvoRubrics, SCORE, and RubricEM update rubrics adversarially or from a rubric buffer within training [50–52]. The groups part on human calibration: among the methods that move the gate with the policy, only RLPR, whose reward is a probability rather than a rubric, reports agreement with human labels, and no method that co-evolves a rubric or judge does (Section S6.6 gives the tally). One study calibrates its verifier against language-model golden judges rather than people [44], and a survey of rubric evaluation names the dynamic regime as the trend [53].

This line established the gate constructions for judged signals: checklists, references, pairwise judges, and rubric buffers. The setting it assumes is weight-space optimization with the policy as the learner. The condition that changes here is the learner: when the scaffold is what learns, the rubric is itself a scaffold coordinate, so the fixed-versus-co-evolved distinction becomes the question of whether a gate under edit can be calibrated at all. The article borrows the constructions and the distinction; it does not score the methods.

S7.4 Governance: lineage, access control, rollback, and promotion across organizations

Three lines bear on the four-property audit. The first is governance surveys. Ding et al. [91] code 435 works on six dimensions per state item, including provenance and recoverability, and find the literature "concentrates more heavily on accumulating and retrieving state than on governing, recovering, or relinquishing it". Lin et al. [92] organize memory security by lifecycle phase and name five primitives, among them principal-scoped retrieval and rollbackability. Neither carries a promotion dimension, and neither scores whether an inner loop fires from session signal.

The second line is the systems that come closest to the four properties. FederatedSkill exchanges semantic skill diffs between clients that are agent instances under one operator, and a server-side evolution agent applies, adapts, or rejects each patch per client while writing an auditable decision log of source patch, reward, and justification; it reports up to 44.4% higher success than isolated self-evolution across 20 task families [18]. EvolveNet broadcasts a shared harness, optimizes it locally at each client, and merges only when a merge fixes at least as many held-out items as it breaks, rolling back to the previous shared harness after a failed round; its authors describe clients as spanning users and organizations, evaluate on benchmark partitions, and note that a program edit can itself encode client information [93]. The registries supply the remaining properties without an inner loop: EDDOps promotes through automatic evaluation thresholds and human sign-off with supersession lineage and emergency rollback, and does not implement role-based access control

[94]; Agentic Configuration Management keeps immutable revisions and baselines that enable rollback [95]; Governed Capability Evolution gates activation and rolls back on monitoring, recovering in 72.4% of drift cases in its proof of concept [96]; and the MCP control planes add role-based access and quarantine without an evaluation gate [97–99]. Letta's Context Repositories commit every memory edit to git and push shared memory across agents without a gate [100].

The third line is federated learning. FedAvg averages client updates in parameter space [101], DP-FedAvg adds user-level differential privacy [102], the field's open problems are catalogued in Kairouz et al. [103], and PromptFL moves the federated object from weights to a shared prompt that participants learn cooperatively [104]. The setting is numeric updates that are comparable by construction. The condition that changes here is the artifact: scaffold promotion is selection and merge over symbolic objects, and the nearest systems aggregate program edits with a language model rather than by averaging, which is why the article uses the federated analogy only as a limit.

S7.5 What is and is not new

The ingredients have precedent, and the article says so. The six substrates are the inventory of the self-evolving-agent, memory, context, skill, and harness surveys, and the memory taxonomy is CoALA's. The gate as a development-set score is the prompt-optimization lineage; the scaffold as an optimizable object is agent symbolic learning, STOP, and workflow search; the persisted artifact that closes Loop 1 is Reflexion, Voyager, and the agent-memory line; the admission rule with a margin and a check is Zhou et al.'s; the gate constructions for judged signals are the rubric-reward line; provenance, recoverability, and rollback as governance dimensions are Ding et al. and Lin et al.; federated aggregation of artifact patches is FederatedSkill and EvolveNet. What is new is the combination and the measurement: a corpus of 256 systems scored on one rubric and released; a classification of each by which of two loops it closes and under which gate type; a classification of the signal that feeds the gate into four verifiability classes, with the demonstration that the class fixes the reachable cells of the design space and the dominant failure; and an audit of every system against four governance properties, with the finding that none meets all four and that the two nearest, FederatedSkill and EvolveNet, each narrow different properties without meeting the set. These are the claims of the abstract and the introduction. Three limits bound them: scoring is single-rater over public evidence of uneven strength (Section V), the corpus closes at the October 2026 cutoff, and the evidence is the published behavior of the 256 systems rather than new experiments (Section I).

References

[1] M. L. Arbuzov, L. Mosbacker, S. Bei, Z. Dong, D. Kalaev, and A. Shvets, "The architecture of errors: From universal impossibility to patch-local LLM reliability," *arXiv preprint arXiv:2605.30628*, 2026.

[2] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[4] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou, "TextGrad: Automatic "differentiation" via text," *arXiv preprint*, 2024.

[5] Y. Yang, Z. Gong, W. Huang, Q. Yang, Z. Zhou, Z. Huang, Y. Li, X. Gao, Q. Dai, B. Liu, K. Qiu, Y. Yang, D. Chen, X. Yang, and C. Luo, "SkillOpt: Executive strategy for self-evolving agent skills," arXiv preprint arXiv:2605.23904, 2026. [Online]. Available: https://microsoft.github.io/SkillOpt

[6] D. Tran and D. Kiela, "Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets," *arXiv preprint*, 2026.

[7] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] Q. Zhang, C. Hu, S. Upasani *et al.*, "Agentic context engineering: Evolving contexts for self-improving language models," arXiv preprint arXiv:2510.04618, 2025.

[9] X. Liu *et al.*, "SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support," *arXiv preprint*, 2026, accepted at ACM SIGIR 2026 Industry Track.

[10] X. Wang *et al.*, "AutoAgent: Evolving cognition and elastic memory orchestration for adaptive agents," *arXiv preprint*, 2026.

[11] P. Xia *et al.*, "SkillRL: Evolving agents via recursive skill-augmented reinforcement learning," *arXiv preprint*, 2026.

[12] J. Zhang *et al.*, "Darwin Gödel machine: Open-ended evolution of self-improving agents," *arXiv preprint*, 2025.

[13] J. Chu, J. S. Lee, J. Park, D. Ko, J. Na, S. Lee, T. Lee, M. Yoon, M. Joo, Y. Xiong, and H. J. Kim, "Retrieval-augmented skill optimization via cross-harness adaptation," *arXiv preprint arXiv:2609.38024*, 2026.

[14] J. Qi, Y. He, J. Liu, Q. Huang, S. Hu, Z. Dang, G. Chen, R. Yang, W. Jiang, Y. Liu, T. Lyu, and F. Li, "Vace: Validation-gated alternating co-evolution of agent models and harnesses," *arXiv preprint arXiv:2609.37105*, 2026.

[15] A. Qu, H. Zheng, Z. Zhou, Y. Yan, Y. Tang, S. Y. Ong, F. Hong, K. Zhou, C. Jiang, M. Kong, J. Zhu, X. Jiang, S. Li, C. Wu, B. K. H. Low, J. Zhao, and P. P. Liang, "CORAL: Towards autonomous multi-agent evolution for open-ended discovery," *arXiv preprint*, 2026.

[16] C. Ling, P. Chen, A. Guan, J. Qu, S. A. Akbar, M. Gopinathan, and E. Cornejo, "PACE: Two-timescale self-evolution for small language model agents," *arXiv preprint*, 2026.

[17] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn, "Meta-Harness: End-to-end optimization of model harnesses," *arXiv preprint*, 2026.

[18] J. Yang, G. Yao, Y. Zhang, R. R. Kompella, G. Liu, and S. Chang, "FederatedSkill: Federated learning for agentic skill evolution," *arXiv preprint arXiv:2606.03143*, 2026.

[19] Z. Ma, S. Yang, Y. Ji, X. Wang, Y. Wang, Y. Hu, T. Huang, and X. Chu, "SkillClaw: Let skills evolve collectively with agentic evolver," *arXiv preprint arXiv:2604.08377*, 2026.

[20] Y. Su, W. Xu, C. Zuo, and E. Bertino, "ChronoMem: Version control and semantic rollback for large language model agent memory," *arXiv preprint arXiv:2607.27773*, 2026.

[21] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," *arXiv preprint*, 2025.

[22] J. Ni *et al.*, "Trace2Skill: Distill trajectory-local lessons into transferable agent skills," *arXiv preprint*, 2026.

[23] S. Alzubi *et al.*, "EvoSkill: Automated skill discovery for multi-agent systems," *arXiv preprint*, 2026.

[24] W. Song, J. Yue, and Z. Pang, "ABSTRAL: Automatic design of multi-agent systems through iterative refinement and topology optimization," *arXiv preprint*, 2026.

[25] Y. He *et al.*, "EvoTest: Evolutionary test-time learning for self-improving agentic systems," *arXiv preprint*, 2025.

[26] Y. Shen, B. Li, and X. Zhang, "SkillOpt-Lite: Better and faster agent self-evolution via one line of vibe," *arXiv preprint arXiv:2607.03451*, 2026.

[27] R. Tiwari, K. Sareen, L. A. Agrawal, J. E. Gonzalez, M. Zaharia, K. Keutzer, I. S. Dhillon, R. Agarwal, and D. Khatri, "Learning, fast and slow: Towards LLMs that adapt continually," *arXiv preprint*, 2026.

[28] S. Dohare *et al.*, "Loss of plasticity in deep continual learning," *Nature*, vol. 632, no. 8026, pp. 768–774, 2024.

[29] N. Vassilyev, W. Berrios, R. Zhang, B. Han, D. Kiela, and S. Mehri, "Reflective context learning: Studying the optimization primitives of context space," *arXiv preprint*, 2026.

[30] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, "Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses," *arXiv preprint*, 2026.

[31] Q. Cai, Y. Zhang, X. Jia, H. Zheng, W. Xue, J. Song, X. Tian, and Y. Guo, "MOSS: Self-evolution through source-level rewriting in autonomous agent systems," *arXiv preprint*, 2026.

[32] T. Zehle, "CANTANTE: Optimizing agentic systems via contrastive credit attribution," *arXiv preprint*, 2026.

[33] K. T.-i. Ong, M. Kang, D. Choi, J. Cho, S. Kim, S. Lim, G. Jang, M. Oh, B. Jeong, S. Kim, T. Kwon, and J. Yeo, "Towards direct evaluation of harness optimizers via priority ranking," *arXiv preprint*, 2026.

[34] M. Nguyen, W. C. Tan, N. A. Hassan, A. Raman, L. H. Lim, and A. F. Razak, "Where does harness-optimization value live? Localized gains and the budget-splitting trap in self-evolving LLM agents," *arXiv preprint arXiv:2609.02889*, 2026.

[35] M. M. Tanjim, J. Subramanian, X. Chen, B. Kveton, S. Mukherjee, A. Zhang, S. Kim, S. Sarkhel, and S. Choudhury, "MOCHA: Multi-objective chebyshev annealing for agent skill optimization," *arXiv preprint*, 2026.

[36] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, "Ratchet: A minimal hygiene recipe for self-evolving LLM agents," *arXiv preprint*, 2026.

[37] H. Liu, H. Yang, T. Jiang, B. Tang, F. Xiong, Y. Luo, and Z. Li, "SkillsVote: Lifecycle governance of agent skills from collection, recommendation to evolution," *arXiv preprint arXiv:2605.18401*, 2026.

[38] Y. Shalev, Z. Ding, and M. Jamnik, "Training language agents to learn from experience," *arXiv preprint*, 2026.

[39] H. Li, R. He, Q. Zhang, C. Ji, Q. Mang, X. Chen, L. A. Agrawal, W.-L. Liao, E. Yang, A. Cheung, J. Zou, K. Olukotun, I. Stoica, and J. E. Gonzalez, "Combee: Scaling prompt learning for self-improving language model agents," *arXiv preprint*, 2026.

[40] A. Gunjal, A. Wang, E. Lau, V. Nath, Y. He, B. Liu, and S. Hendryx, "Rubrics as rewards: Reinforcement learning beyond verifiable domains," *arXiv preprint arXiv:2507.17746*, 2025.

[41] R. Jia, Y. Yang, Y. Gai, K. Luo, S. Huang, J. Lin, X. Jiang, and G. Jiang, "Writing-zero: Bridge the gap between non-verifiable tasks and verifiable rewards," *arXiv preprint arXiv:2506.00103*, 2025.

[42] K. Shi, Y. Liu, P. Wang, A. R. Fabbri, S. Joty, and A. Cohan, "References improve LLM alignment in non-verifiable domains," *arXiv preprint arXiv:2602.16802*, Feb. 2026.

[43] X. Jin, J. Zhang, T. Cao, and B. Yang, "Beyond score prediction: LLM-based essay scoring and feedback generation via reinforcement learning with rubric rewards," *arXiv preprint arXiv:2607.19219*, Jul. 2026.

[44] A. Plesner, C. Northcutt, F. Guzmán, and A. Athalye, "A cheap verifier is good enough: LLM post-training is robust to erroneous rewards," *arXiv preprint arXiv:2609.33467*, Sep. 2026.

[45] M. Bhattarai, I. Boureima, N. R. Ranasinghe, S. Pakin, and D. O'Malley, "Rubric-grounded RL: Structured judge rewards for generalizable reasoning," *arXiv preprint arXiv:2605.08061*, May 2026.

[46] R. Jia, Y. Yang, W. Wang, Y. Wu, Y. Gai, S. Tao, M. Zhou, J. Lin, X. Jiang, and G. Jiang, "Open rubric system: Scaling reinforcement learning with pairwise adaptive rubric," *arXiv preprint arXiv:2602.14069*, Feb. 2026.

[47] C. Lv, J. Zhou, W. Zhao, J. Xu, S. Dou, Z. Huang, M. Tian, X. Wang, Z. Guo, Y. Liu, P. Zhou, T. Gui, L. Tian, X. Zhou, X. Zheng, X. Huang, and J. Zhou, "Learning query-specific rubrics from human preferences for DeepResearch report generation," *arXiv preprint arXiv:2602.03619*, Feb. 2026.

[48] T. Yu, B. Ji, S. Wang, S. Yao, Z. Wang, G. Cui, L. Yuan, N. Ding, Y. Yao, Z. Liu, M. Sun, and T.-S. Chua, "RLPR: Extrapolating RLVR to general domains without verifiers," *arXiv preprint arXiv:2506.18254*, Jun. 2025.

[49] R. Xu, T. Liu, Z. Dong, T. Yu, I. Hong, C. Yang, L. Zhang, T. Zhao, and H. Wang, "Alternating reinforcement learning for rubric-based reward modeling in non-verifiable LLM post-training," *arXiv preprint arXiv:2602.01511*, 2026.

[50] H. Ding, B. Huang, Y. Fang, W. Liao, Z. Li, J. Zhang, Z. Wu, J. Zhao, and Y. Wang, "EvoRubrics: Dynamic rubrics as rewards via adversarial co-evolution for LLM reinforcement learning," *arXiv preprint arXiv:2606.23038*, Jun. 2026.

[51] H. Zhu, C. Cai, Y. Song, X. Chen, S. Han, and Y. Guo, "Self-evolving deep research via joint generation and evaluation," *arXiv preprint arXiv:2606.04507*, Jun. 2026.

[52] G. Li, B. Dalvi Mishra, Z. Wang, J. Yan, Y. Chen, C.-L. Li, L. T. Le, R. Han, G. Lee, H. Tong, C.-Y. Lee, and T. Pfister, "RubricEM: Meta-RL with rubric-guided policy decomposition beyond verifiable rewards," *arXiv preprint arXiv:2605.10899*, May 2026.

[53] H. Chen, Z. Han, Y. Yan, Q. Zhu, M. Sun, and W. Che, "From holistic evaluation to structured criteria: Rubrics across the evolving LLM landscape," *arXiv preprint arXiv:2606.08625*, 2026.

[54] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan *et al.*, "A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence," *Transactions on Machine Learning Research*, 2026, arXiv:2507.21046v4, 16 Jan 2026.

[55] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng, "A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems," *arXiv preprint arXiv:2508.07407*, Aug. 2025.

[56] H. Zhou, H. Hu, T. Luo, Y. Shang, C. Fang, Z. Chen, L. Xiao, and Q. Zhang, "Self-evolving coding agents," *arXiv preprint arXiv:2608.03392*, Aug. 2026.

[57] Z. Ren, Y. Chen, D. Guo, G. Rong, T. Li, R. B. Xiong, Q. Lan, W. Wang, L. Nanbo, Y. Yang, M. Zhuge, and J. Schmidhuber, "Self-improvements in modern agentic systems: A survey," *arXiv preprint arXiv:2607.13104*, 2026.

[58] P. Du, "Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers," *arXiv preprint arXiv:2603.07670*, Mar. 2026.

[59] W.-C. Huang, W. Zhang, Y. Liang, Y. Bei, Y. Chen, T. Feng *et al.*, "A survey of agent memory in the second half: Towards self-evolving and long-horizon agents," *arXiv preprint arXiv:2602.06052*, 2026.

[60] J. Luo, Y. Tian, C. Cao, Z. Luo, H. Lin, K. Li, C. Kong, R. Yang, and J. Ma, "From storage to experience: A survey on the evolution of LLM agent memory mechanisms," *arXiv preprint arXiv:2605.06716*, May 2026.

[61] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, "A survey of context engineering for large language models," *arXiv preprint arXiv:2507.13334*, Jul. 2025.

[62] Q. Hua, L. Ye, D. Fu, Y. Xiao, X. Cai, Y. Wu, J. Lin, J. Wang, and P. Liu, "Context engineering 2.0: The context of context engineering," *arXiv preprint arXiv:2510.26493*, Oct. 2025.

[63] Y. Zhou, W. Shu, Y. Su, W. Du, Y. Fang, and X. Lin, "A comprehensive survey on agent skills: Taxonomy, techniques, and applications," *arXiv preprint arXiv:2605.07358*, 2026.

[64] K. Ding, Y. Zhou, C. Jin, F. Tong, M. Zhou, and D. N. Metaxas, "Agent skill evaluation and evolution: Frameworks and benchmarks," *arXiv preprint arXiv:2606.11435*, Jun. 2026.

[65] B. Xia, L. Zhu, Z. Xing, Q. Lu, D. Sejdinovic, and X. Xu, "Harnessing agent skills: Architectural patterns and a reference architecture for skill-mediated LLM agents," *arXiv preprint arXiv:2606.20631*, May 2026.

[66] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," *arXiv preprint arXiv:2211.01910*, 2022.

[67] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, and M. Zeng, "Automatic prompt optimization with "gradient descent" and beam search," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 2023, arXiv:2305.03495.

[68] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.03409.

[69] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, "EvoPrompt: Connecting LLMs with evolutionary algorithms yields powerful prompt optimizers," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.08532.

[70] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," *arXiv preprint arXiv:2309.16797*, 2023.

[71] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714*, 2023.

[72] K. Opsahl-Ong, M. J. Ryan, J. Purtell, D. Broman, C. Potts, M. Zaharia, and O. Khattab, "Optimizing instructions and demonstrations for multi-stage language model programs," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, 2024, arXiv:2406.11695.

[73] C.-A. Cheng, A. Nie, and A. Swaminathan, "Trace is the next AutoDiff: Generative optimization with rich feedback, execution traces, and LLMs," *arXiv preprint arXiv:2406.16218*, 2024.

[74] W. Zhou, Y. Ou, S. Ding, L. Li, J. Wu, T. Wang, J. Chen, S. Wang, X. Xu, N. Zhang, H. Chen, and Y. E. Jiang, "Symbolic learning enables self-evolving agents," *arXiv preprint arXiv:2406.18532*, 2024.

[75] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai, "Self-taught optimizer (STOP): Recursively self-improving code generation," in *First Conference on Language Modeling (COLM 2024)*, 2024, arXiv:2310.02304.

[76] M. Robeyns, M. Szummer, and L. Aitchison, "A self-improving coding agent," *arXiv preprint arXiv:2504.15228*, 2025.

[77] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "Language agents as optimizable graphs," in *Forty-first International Conference on Machine Learning (ICML 2024)*, 2024, arXiv:2402.16823.

[78] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," *arXiv preprint arXiv:2410.10762*, 2024.

[79] J. S. Park *et al.*, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[80] G. Wang *et al.*, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint*, 2023.

[81] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," in *Proceedings of the 38th Annual AAAI Conference on Artificial Intelligence (AAAI-24)*, 2024, arXiv:2308.10144.

[82] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[83] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, "Agent workflow memory," *arXiv preprint arXiv:2409.07429*, 2024.

[84] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-MEM: Agentic memory for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS 2025)*, 2025, arXiv:2502.12110.

[85] M. Suzgun, M. Yuksekgonul, F. Bianchi, D. Jurafsky, and J. Zou, "Dynamic cheatsheet: Test-time learning with adaptive memory," *arXiv preprint arXiv:2504.07952*, 2025.

[86] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents (CoALA)," *Transactions on Machine Learning Research (TMLR)*, 2023.

[87] J. Guo, Z. Hao, C. Wang, C. Fan, T. Luo, H. Li, Y. Gao, H. Mei, J. Peng, R. Xu, M. Dong, H. Wu, M. Zheng, K. Han, S. Wang, C. Xu, and Y. Wang, "From question answering to task completion: A survey on agent system and harness design," *arXiv preprint arXiv:2606.20683*, Jun. 2026.

[88] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS 2023), Datasets and Benchmarks Track*, 2023, arXiv:2306.05685.

[89] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.

[90] S. Shankar, J. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran, and I. Arawjo, "Who validates the validators? aligning LLM-assisted evaluation of LLM outputs with human preferences," *arXiv preprint arXiv:2404.12272*, 2024.

[91] T. Ding, A. Nannapaneni, B. Liu, and L. Zhang, "Always-on agents: A survey of persistent memory, state, and governance in LLM agents," *arXiv preprint arXiv:2606.30306*, 2026.

[92] Z. Lin, X. Hao, R. Fu, S. Cui, K. Chen, C. Li, Z. Li, and F. Xiong, "A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle," in *Proceedings of EMNLP 2026*, 2026, arXiv:2604.16548v3, 22 Sep 2026.

[93] J. Nie, Y. Zhang, Q. Cai, Y.-m. Cheung, X. Tian, and B. Han, "Evolvenet: Collaborative harness evolution for agent self-improvement," *arXiv preprint arXiv:2608.04968*, 2026.

[94] R. Kang and V. Wang, "Registry-governed agent lifecycle: Completing EDDOps with evaluation-driven registration, promotion, and retirement on AWS AgentCore," *arXiv preprint arXiv:2607.00345*, Jul. 2026.

[95] A. Quessada-Vial, "Agentic configuration management (ACM): A reference configuration model for governed agentic systems," *arXiv preprint arXiv:2608.11166*, Aug. 2026.

[96] X. Qin, S. Luan, J. See, Z. Boukhers, C. Yang, and Z. Li, "Governed capability evolution: Lifecycle-time compatibility checking and rollback for AI-component-based systems, with a proof-of-concept evaluation on embodied agents," *arXiv preprint arXiv:2604.08059*, Apr. 2026.

[97] agentic-community, "Mcp gateway & registry," https://github.com/agentic-community/mcp-gateway-registry, 2026.

[98] Microsoft, "Inventory and discover mcp servers in your api center," https://learn.microsoft.com/en-us/azure/api-center/register-discover-mcp-server, 2026.

[99] TrueFoundry, "Mcp gateway authentication and security," https://www.truefoundry.com/docs/ai-gateway/mcp-gateway-auth-security, 2026.

[100] Letta, "MemFS; memory & dreaming; introducing context repositories: Git-based memory for coding agents," https://docs.letta.com/letta-code/memfs, https://docs.letta.com/letta-agent/memory, https://www.letta.com/blog/context-repositories/, 2026.

[101] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54, 2017, pp. 1273–1282.

[102] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *International Conference on Learning Representations (ICLR)*, 2018, arXiv:1710.06963.

[103] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis *et al.*, "Advances and open problems in federated learning," *arXiv preprint arXiv:1912.04977*, 2019, published in Foundations and Trends in Machine Learning, vol. 14, no. 1–2, 2021.

[104] T. Guo, S. Guo, J. Wang, and W. Xu, "PromptFL: Let federated participants cooperatively learn prompts instead of models – federated learning in age of foundation model," *arXiv preprint arXiv:2208.11625*, 2022.